

AssistOS

THE OPERATING LAYER FOR
SOVEREIGN INTELLIGENCE



Copyright

Copyright © 2026 Axiologic Research
All rights reserved.

KDP publishing rights held by Outfinity SRL (Iași, Romania).
Available for free on ScriptaHub.com (www.ScriptaHub.com).

Author's Note

This work was created under the umbrella of Axiologic Research as part of two interconnected research programs, one dedicated to Artificial Intelligence and the other to Outfinitism, both led by Sînică Alboaie. To explore the details of how these two programs intersect and build upon each other, we invite readers to visit the Outfinity Venture Studio page at www.outfinity.ch.

While Artificial Intelligence language models were used to assist with text generation, the foundational philosophy, thematic structure, and analytical approach derive exclusively from the author's decades of dedicated study of social phenomena, social technologies, and genuine hard technologies resulting from various European and self funded research projects. Recently, within the ACHILLES research project, we have been deeply studying AI generated literature and its automated verification using neuro symbolic approaches; all the free books made available to the public are part of the suite of works we use to test our latest technologies.

About the second edition

AssistOS project website: <https://www.assistos.org/>

This second edition explains why we are building AssistOS and how we envisage people using it. Our starting point is the person at the computer: someone who wants to delegate useful work, follow its progress and retain control of the result. AssistOS brings AI copilots, agents and their personalized Robot environments into a shared workplace. The chapters begin with that experience, then explain the machinery and the wider thesis behind it.

Computing is becoming a collaborative activity. AI changes what people can expect from a computer. Instead of manually translating every intention into a sequence of application commands, a person can describe an outcome, let software perform part of the work and intervene when judgment is needed. The interface must consequently make ongoing work visible: what is being attempted, which files are changing, what needs a decision and what has been completed. AssistOS is designed around this continuing collaboration.

More intelligence can run on equipment we control. We expect language models and other machine-learning models to become increasingly useful on personal computers and private organizational infrastructure. On-device text generation, extraction and tool use already have concrete implementations, including Apple's Foundation Models framework and Microsoft's Phi work for PCs. [E13, E14] The opportunity is to build an environment that benefits from these improvements without requiring every task or private document to pass through a remote provider. Larger remote models can remain an explicit option when the work warrants them.

Precise work needs more than fluent generation. Our architectural thesis is that many dependable enterprise agents will combine a relatively small learned component with a substantial layer of conventional software and symbolic processing. Models interpret requests or propose candidates; code, schemas, rules, constraints and verifiers perform the operations that must be exact. Neuro-symbolic systems can progressively take over bounded work otherwise assigned to larger models. AlphaGeometry illustrates this division in a specialized domain by combining a language model with symbolic deduction. [E15] It does not establish that every enterprise task is solved. It does show why model size alone is an incomplete account of capability.

Trustworthiness, efficiency and privacy can advance together. When a local model extracts a proposed invoice record, ordinary software can verify the fields and calculate the total, while explicit rules decide whether the result is acceptable. This division can reduce unnecessary generation, make errors easier to locate and keep sensitive records local. We therefore see trustworthy AI, efficient computation and privacy as connected engineering objectives. Their benefits must be measured together: a system is not improved if it saves inference cost while increasing mistakes, review effort or data exposure.

An open local alternative matters to citizens and companies. Linux demonstrates the lasting importance of an operating foundation that can be inspected, adapted and operated independently. We consider a substantial open ecosystem for local AI highly likely, and AssistOS is intended to contribute to it. People should be able to use capable AI without making a large company's cloud the compulsory custodian of their work. For a citizen this concerns private writing, records and research; for a company it concerns confidential knowledge, operational independence and continuity. Sovereignty means having a workable choice of where and under whose control computation happens.

The operating-system interface can become an IDE for everyday work. An integrated development environment, or IDE, gives programmers a coherent place for editing, running, inspecting, testing and revising their work. AssistOS aims to extend that approach to any computer-based activity. Documents, spreadsheets, research, administration and creative projects can benefit from version history, reviewable changes, reproducible procedures and isolated experiments. People should not need to become programmers to use these practices. AI copilots interpret their intentions, agents perform operations, and Robots provide the persistent environments in which that work proceeds.

A Robot is a personalized virtual employee. An AI Agent is software, such as a coding agent. The same software can serve several purposes through different configurations, accounts or subscriptions. AssistOS calls each persistent, personalized working environment a Robot: a named role with its own settings, permitted resources, history and private execution space. One person might configure a Research Robot and a Software Reviewer using the same underlying agent while giving them different projects and accounts. These environments are first-class AssistOS objects, so changing an agent or model need not erase the worker's continuity.

Documents and meetings should include Robots by design. We envisage an office-like environment in which humans and Robots work on the same documents, tables, diagrams and other artifacts. Video conferences should also support Robot participants from the outset. A team could ask a Robot to check a figure, retrieve evidence or prepare a document change during a meeting, then review the result together. Explorer, WebChat and WebMeet express this common design: the conversation, the shared object and the work being performed belong to the same project.

Isolation should be part of ordinary installation and use. Agents increasingly execute code, install dependencies and operate applications. AssistOS aims to make installing such capabilities in sandboxes straightforward, using Ploinky's packaging and runtime mechanisms. [P8] Each Robot should also have its own private sandboxed environment, with separate writable state, credentials and permitted resources. Several Robots may reuse the same installed software without sharing one unrestricted working environment. This is how the virtual-employee model becomes operational: each worker has a place to work and an explicit boundary around what it can affect.

This is a product and research direction, with useful building blocks already available. The scenarios in this book describe the intended integrated experience; what a particular installation can do depends on its packages and integrations. The practical chapters show how to turn the thesis into a first assignment, a reviewable result and a method that can be reused. [P1, P8, P9]

Contents

Chapter 1 Working with your first Robot

- Start with one project and one useful assignment
- A working day with a RoboTeam
- Distinguish the Robot from its capabilities
- Delegate work and return to the result
- Watch a Robot use a computer or browser
- Guide the work while it is running

Chapter 2 Organizing the shared workplace

- Find projects and artifacts in Explorer
- Assign and continue tasks through WebChat
- Work together in WebMeet
- Preserve artifacts and task histories
- Keep approved work distinct from proposals and derived results
- Adapt the workplace with plugins

Chapter 3 Applying AssistOS to useful work

- Develop and review software
- Investigate questions and preserve research evidence
- Operate websites and administrative applications
- Create documents and shared creative work
- Use the same pattern for personal and professional collections
- Run a pilot and measure its value

Chapter 4 Making delegated work repeatable and controllable

- Turn a successful task into a reusable procedure
- Improve the procedure using failures and corrections
- Define the execution boundary
- Set the scope of delegation
- Manage Robot identity and persistent sessions
- Inspect actions and recover from failure
- Govern Robots across an organization

Chapter 5 Understanding the machinery that supports the work

- Provide the tools that a model needs
- Package a deployable capability
- Give each Robot a persistent home
- Connect tools through suitable interfaces
- Choose where the work runs

Chapter 6 Developing the wider platform

- Understand AssistOS as an operating layer
- Preserve control and continuity
- Develop a useful capability catalog
- Build commercial value around deployment and maintenance
- Assess the strategic proposition
- Test the research directions against real work

Appendix A Glossary

- Core AssistOS terms
- General AI and software terms

Appendix B Architecture at a glance

The smallest useful architecture

A company deployment

Appendix C Questions for adoption

Questions for a first workflow

Questions for an organization or investor

Sources and further reading

AssistOS and Ploinky sources

External technical references

Introductory glossary references

How to use this handbook

AssistOS is a proposed workplace for persistent virtual employees called Robots. A Robot has a role, a home, access to selected capabilities and a history of tasks. People assign the work, guide it when necessary and review the resulting artifacts. The project preserves the files and decisions needed to continue later. This model is useful for a single person as well as a team.

Start with Chapters 1 to 3 to understand the daily experience and choose a first workflow. Chapter 4 develops repeatable procedures, evaluation and control. Chapter 5 explains the supporting runtime and integration choices. Chapter 6 examines the operating-layer thesis, distribution, commercial development and a short research horizon. The appendices provide definitions and practical questions for a deployment.

Robot, AI Agent and Ploinky Agent have distinct meanings. The Robot is the persistent worker a user delegates to. An AI Agent is software that reasons and acts. A Ploinky Agent is a deployable package that can contain such software, ordinary tools or a service. The user can work mainly with Robots, projects and results; the technical distinctions become useful when configuring or replacing capabilities.

Chapter 1 Working with your first Robot

Begin with a bounded assignment and a result you can inspect. The same relationship then supports several Robots, background work and live supervision.

Start with one project and one useful assignment

The first useful AssistOS experience should be small enough to understand from beginning to end. You create a project, configure a Robot for a role, give it access to the relevant material and assign a task with an observable result. You can then leave it working, inspect progress when necessary and review what it produces. A team of Robots becomes useful after this basic relationship works.

Consider a consultant preparing a comparison of three supplier proposals. The project contains the proposals, a short requirements document and a Results folder. The consultant creates a Research Robot whose role is to extract and compare information from those documents. It receives document-reading and writing capabilities, the consultant's preferred report language and access to this project. It does not need a simulated personality or a lengthy biography. The useful configuration describes its work, available tools and working context.

The task could read: Compare the three proposals against requirements.docx. Produce a Word report in Results, with one comparison table, a short explanation of the material differences and references to the relevant proposal pages. Keep prices in their stated currencies. Mark missing information as missing. Preserve the source documents. Prepare the comparison for my review.

Table 1. A first task has a small set of explicit decisions

Decision	Supplier comparison example
Project	Three proposals, requirements and a Results folder
Robot role	Read documents and prepare a comparison
Working scope	The selected project and its output folder
Expected result	Word report with comparable fields and page references
Human decision	Accept the comparison or request a correction
Reusable record	Inputs, criteria, procedure, output and review history

This instruction gives the Robot enough direction to create an inspectable result. If two proposals quote annual prices and one quotes a monthly price, the Robot should show the basis of comparison. If the contract period is absent, it should identify the uncertainty instead of inventing a total. The consultant can answer a focused question while the task retains the work already completed.

During execution, the useful screen shows the active task, its working folder and its status. A short progress update might say that two proposals have been processed and the third contains scanned pages that require another extraction method. That is enough to decide whether to wait, supply a clearer file or change the scope. Detailed execution information remains available when diagnosing a problem.

At review, the consultant opens the report and follows a few references back to the proposals. A plausible-looking table is insufficient if its figures cannot be checked. The consultant accepts the comparison, requests corrections to a specific row or rejects an unsupported recommendation.

Acceptance applies to this result and this version. The Robot's completion message does not itself decide which supplier to select.

The project now contains reusable work: the original proposals, comparison criteria, report, corrections and decision. Next month the consultant can reuse the same Robot and procedure with new inputs. The continuity belongs to the project and the configured role, even if a different model performs the next comparison.

This is a worked example of the intended interaction, rather than a sequence of fixed interface buttons. Its availability depends on the document tools, Robot configuration and workspace features installed in a particular deployment. The remaining chapters explain how to extend this pattern to background tasks, live applications, meetings and whole teams.

A working day with a RoboTeam

The same pattern can extend to a small engineering team preparing a customer release. This worked scenario combines background execution, a browser task and a meeting. The team has an application repository, a test environment, a backlog of issues, a browser-only vendor portal and release notes. Its aim is to give each Robot a distinct piece of work while people retain the shared release decision.

In AssistOS, the team can create a small RoboTeam. One Robot is configured as a Build Engineer. Another is a Web Operator. A third is a Research and Documentation Robot. These names are not claims that the software possesses a human profession in any legal or psychological sense. They are roles that make delegation understandable. Each Robot has a persistent home, comparable in spirit to a user home directory on Linux. The home can contain its configuration, preferences, cached state, agent-specific files and, when policy allows, a browser profile. The Robot also has controlled access to project folders and to the Ploinky Agents that provide its actual capabilities.

The product story begins when a person gives the Build Engineer a task in the AssistOS Copilot, exposed through WebChat: “Prepare the release candidate. Work only in the release branch. Run the test suite, fix failures that are clearly local to this branch, and stop if you need a production credential.” The task becomes visible as work, not merely as a message. The Robot receives a working directory and launches the coding agent configured for that role. That agent might be Codex, Claude Code, OpenHands, a proprietary engineering agent, or another tool entirely. The coding agent is not the Robot. It is software installed inside, or made available through, a Ploinky Agent that the Robot uses to perform the task.

The human does not need to remain in the chat. The Robot can work in the background. It reads the repository it was allowed to read, makes changes in the task folder, runs permitted commands and records progress. If the task succeeds, the user sees the resulting artifacts and a report. If the task fails, the failure should be a first-class outcome: the Robot reports where it stopped, what it tried and what it needs. A useful virtual worker is not defined by never failing. It is defined by failing legibly enough that a human can decide what to do next.

At the same time, the Web Operator receives a different kind of assignment. The vendor portal has no practical API. It is a normal website used by humans. The Robot opens its browser environment using its persistent browser profile and navigates to the portal. The person can watch that browser session in real time. If a login challenge appears, the user can take control. If the website shows an unexpected legal confirmation, the Robot can stop and request intervention. When the human is finished, control returns to the Robot. The objective is not to pretend that the AI is perfectly autonomous. The objective is to make the boundary between automation and human judgment easy to cross.

The Research and Documentation Robot is asked to update the release notes from the merged changes and verify three technical claims against external documentation. It can do this as background work, but later it joins the noon WebMeet session. During the meeting, humans discuss a last-minute compatibility issue. The Robot can access the approved project context, search the evidence it collected and contribute its results through the same collaboration surface. If the meeting uses a shared screen or shared blackboard, the Robot can be asked to modify the shared state just as another participant might annotate a diagram. The meeting becomes a place where conversation can turn directly into controlled execution.

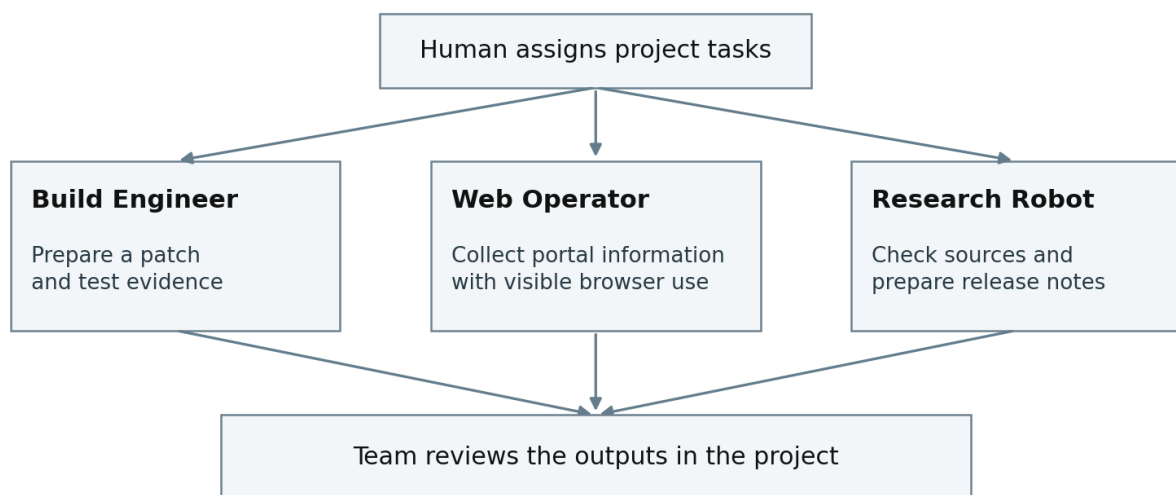


Figure 1. A day with a RoboTeam: background work, browser operation and meeting collaboration can use the same persistent Robot model.

Figure 1 illustrates why the AssistOS story starts with a team rather than a model. The user does not experience a chain of infrastructure products. The user experiences roles, tasks, visible activity and results. One Robot may work for forty minutes without supervision. Another may need three brief interventions because the external website is unpredictable. A third may be most valuable when it remains present during a collaborative session and converts decisions into artifacts. These are different modes of work, but they belong to one mental model.

The model also changes how an organization thinks about responsibility. A background coding task should have an owner, a scope and an output location. A browser task should have explicit rules about which domains, credentials and actions are allowed. A Robot participating in a meeting should have a clear role and should not silently gain access to every object visible to every human participant. The virtual-employee metaphor is useful only if it leads to better boundaries, not if it encourages anthropomorphism and vague authority.

A Monday-morning view of AssistOS therefore looks less like a futuristic robot laboratory and more like a work dashboard. There are projects. There are people. There are Robots with names and roles. Some tasks are queued, some are running, some need human input and some have produced results. The important change is not that AI can talk. The change is that AI can occupy a managed place in the organization's flow of work.

Distinguish the Robot from its capabilities

AssistOS separates the worker a person delegates to from the software that performs the work. Three terms make that separation explicit: Robot, AI Agent and Plinky Agent. They name different responsibilities in the system.

The first level is the Robot: a persistent, personalized environment presented as a virtual employee. It has a role, a home, configuration, state, task history and a private sandboxed execution space. It is the thing a user delegates to. When the user says ‘ask the Research Robot to compare these contracts,’ the sentence should make sense without knowing which model, agent framework or container image will execute the request. The identity belongs to the configured worker, not to one running process.

The second level is the AI Agent. This is software that performs reasoning and action. A coding agent is the clearest example because its behavior is easy to observe: it inspects files, plans changes, edits code, executes commands and evaluates results. Codex and Claude Code are examples of this class, but the architecture is intentionally not limited to them. An Agent may specialize in data analysis, browser operation, scientific research, document review or another domain. Some Agents are highly autonomous; some are little more than sophisticated tool-using assistants. From the Robot’s perspective, both can be capabilities.

For example, a person can configure two Robots around the same coding agent. The Implementation Robot uses a development account and can modify a test project; the Review Robot uses another configuration and reads proposed changes. Their model subscriptions or service accounts can also differ where the provider permits that use. Separate settings, credentials, budgets, project access and histories make them different workers even though the executable software is identical. A Robot can in turn use several agents and tools.

The third level is the Ploinky Agent. The name reflects current Ploinky terminology, where an “agent” is broadly a deployable tool. To avoid confusion, this handbook treats a Ploinky Agent as a packaging and runtime concept. It can wrap a command-line program, an MCP server, an OpenAI-compatible endpoint, a long-running web service, a data-processing engine, a coding agent or an entire containerized application with difficult dependencies. The Ploinky Agent says, in effect, “here is a capability that AssistOS knows how to install, start, isolate, expose and manage.”

Table 2. The vocabulary that keeps the architecture understandable

Term or aspect	Meaning or practical use
Robot	The persistent, user-facing virtual worker. It has a role, a home, task history, configuration and controlled authority.
AI Agent	Reasoning-and-action software such as a coding agent, browser agent or research agent. It is a capability, not the durable worker identity.
Ploinky Agent	A deployable capability package. It can contain an AI Agent, deterministic tools, a service, a CLI, an MCP server or a larger containerized stack.
Ploinky	The deployment and runtime system that installs, starts, isolates and exposes Ploinky Agents.
Explorer	The web workspace in which people see projects, files, Robots, tasks and plugins.
WebChat / Copilot	The conversational dispatch surface used to create tasks, route instructions and continue work.
WebMeet	The real-time collaboration surface for people and Robots, including conferencing, screen sharing and shared state.
RoboTeam	A set of Robots configured to work for an individual or team under shared project context and policy.

The table matters because it prevents a common architectural mistake: tying the user’s durable identity to one particular AI program. If the Build Engineer Robot is identical to a specific release of

a coding agent, then replacing that agent effectively destroys the worker. Its settings, policies and history become accidental extensions of a vendor-specific application. AssistOS instead aims to keep the Robot stable while allowing its underlying capabilities to change. The Robot is the role and continuity layer. Ploinky provides the deployable capabilities. AI Agents are among the engines that can run inside those capabilities.

This separation also clarifies why a Robot needs a home. A coding agent may store its own configuration under a home directory. A browser may maintain cookies and local storage. A specialized tool may need cached indexes, downloaded models or domain-specific settings. If every task starts from a completely blank environment, repeated work becomes expensive and frustrating. If every task shares one unconstrained global environment, security and reproducibility deteriorate. A Robot home creates a middle ground: persistent state that belongs to the role, while task-specific folders and permissions can remain bounded.

The home should not be confused with unrestricted memory. Good system design distinguishes persistent configuration from project data, and both from secrets. A Robot may need to remember that it prefers a particular code formatter. It may need a browser profile that preserves a login session. It may need access to an encrypted credential through a secrets mechanism. These are not the same category of state, and the platform should not store them with the same exposure rules merely because they all feel like “memory.” The virtual-worker mental model is valuable precisely because it encourages the system designer to ask what a worker should keep, what a task should receive and what an administrator should control.

The distinction between Robot and Agent also prevents misplaced trust. An Agent can be swapped because a new tool is better at coding or because an enterprise changes model provider. The Robot’s permissions should not automatically expand simply because its underlying Agent has become more capable. A better engine does not deserve a larger security boundary by default. Likewise, a Robot’s role can remain narrow even if it technically has access to a general-purpose agent. Capability and authority are different properties.

For users, the vocabulary can be even simpler. They should mostly see Robots, projects and results. Technical administrators and developers need to understand Ploinky Agents because they package and deploy capabilities. AI researchers and tool builders care about the internal Agent architecture. The platform earns usability by showing each audience the layer where its decisions are meaningful.

Delegate work and return to the result

Background work is where the virtual-employee metaphor becomes materially different from ordinary chat. In a chat interaction, the user usually remains inside the conversational loop. Even if a model calls tools, the interaction is psychologically synchronous: the person asks, waits, inspects and asks again. A Robot task can be asynchronous. The user defines a job, the platform binds the job to a folder and a capability, the Robot works, and the user returns when there is something worth reviewing.

This does not mean that background work should be an unstructured command such as “fix everything.” The quality of delegation matters more as the execution horizon grows. A good background task has an objective, a scope, a working area, stopping conditions and an expected form of evidence. For a coding task, the Robot may be told to work in one branch and to stop before publishing. For a research task, it may be told which source classes are acceptable and where to write notes. For a document task, it may be told to preserve the original and create a revised copy. AssistOS can help formulate these constraints through the Copilot rather than forcing the user to write a formal specification.

The task folder is important. A Robot should be able to work where the relevant artifacts live, but the folder should also be a visible unit of accountability. The user can inspect what changed there. Multiple tasks can use separate working folders to reduce collisions. A coding Robot may use a worktree. A data-analysis Robot may copy a dataset into a task-specific sandbox. A report-writing Robot may create intermediate drafts alongside a source manifest. The exact technique varies by domain, but the principle is stable: autonomous work is safer when its working set is explicit.

A background task also needs progress without demanding constant attention. Progress is not a theatrical stream of chain-of-thought. It is operational state. The Robot is reading sources, running tests, waiting on a dependency, requesting approval, retrying a failed command, or packaging results. These states allow the user to distinguish productive work from a stalled process. A good interface can summarize them without exposing private reasoning traces or flooding the user with low-level logs.

Table 3. Task states should describe work, not hidden reasoning

Term or aspect	Meaning or practical use
Queued	The task is accepted but has not yet received execution capacity.
Running	The Robot is actively using its assigned capabilities.
Needs input	The Robot can continue after a human supplies a decision, file, approval or authentication step.
Blocked	An external dependency, policy or technical condition prevents useful progress.
Completed	Expected artifacts were produced and the task reached its acceptance condition.
Failed	The Robot stopped without satisfying the task and records the reason and available evidence.
Stopped	A human or policy intentionally ended execution before completion.

A task that reaches “Needs input” should preserve its context so the user can answer one question and continue. This sounds obvious, yet many agent systems treat intervention as a failure that destroys the run. AssistOS should treat intervention as a normal control mechanism. The human may choose among options, attach a missing file, approve network access or simply provide a sentence of clarification. The Robot can then resume in the same environment. The cost of supervision falls dramatically when intervention does not require reconstructing the task from the beginning.

Background work is also where economics becomes visible. Long-running agents can consume model tokens, CPU time, GPU capacity, storage and external API calls. A platform for virtual workers needs budgets. The right abstraction is not merely a global monthly quota. A team may want a cost ceiling for a task, a daily budget for one Robot or a policy that uses a local model for routine steps and escalates to a remote model for difficult reasoning. The user should understand the policy in terms of outcomes and limits, while the platform handles the routing details.

The strongest background workflows produce reviewable artifacts. A software Robot returns a patch, test results and a short report. A research Robot returns sources, notes and a draft with traceable claims. A browser Robot returns a transaction record or downloaded document. A data Robot returns a dataset, script and chart. These outputs remain in the project where they can be checked and reused.

The practical value is not that a person can delegate everything. It is that a person can convert waiting time into parallel work. A founder can ask one Robot to investigate a supplier while another

updates a prototype. An engineer can delegate three bounded fixes while reviewing a more difficult architecture change. A researcher can run a literature search while preparing an experiment. The unit of productivity becomes the supervised portfolio of tasks, not the speed at which one person can type prompts.

Watch a Robot use a computer or browser

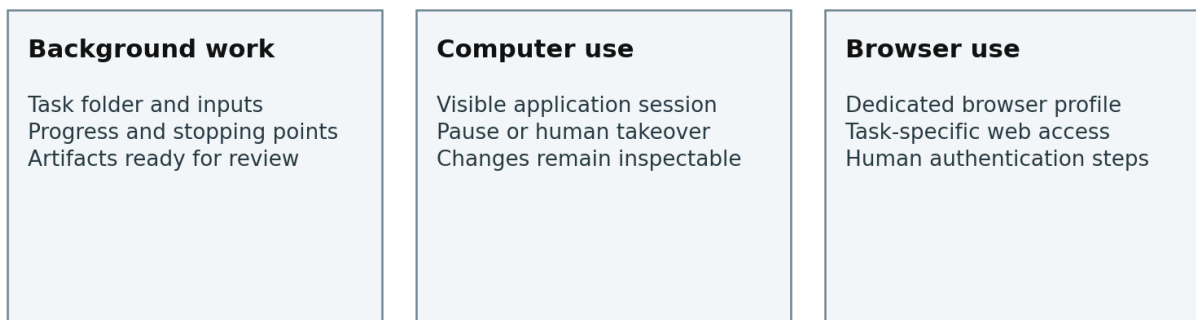
Many economically valuable tasks still live behind graphical interfaces. An accounting package may have a limited API. A scientific instrument may expose a desktop application. A legacy enterprise system may require menus and forms. A website may provide no stable programmatic interface at all. If virtual workers can operate only APIs designed for agents, they can automate only the cleanest part of the world. Computer use extends the capability surface to the software that humans already use.

AssistOS should treat computer use as an observable session. The Robot operates inside a desktop or application environment that can be streamed to the user. The human sees the same screen state the Robot sees. The Robot can click, type, inspect and navigate according to the capabilities of the installed computer-use Agent. When the system reaches a point that requires judgment, the human can intervene. The distinction from remote desktop support is that the Robot is an active operator, while the person becomes the supervisor who can take control when necessary.

The visibility requirement is more than a convenience. Graphical interfaces contain ambiguity that is hard to capture in logs. A button may be disabled. A modal dialog may obscure the application. A website may change its layout. A file picker may open in an unexpected directory. An agent can misinterpret these states. Real-time viewing gives the user a fast way to recognize that the automation has entered the wrong context. The ability to stop the session is therefore a core safety primitive, not a decorative control.

Browser use deserves its own product concept even though a browser is technically a computer application. The browser carries persistent identity in a particularly sensitive form: cookies, local storage, saved sessions, certificates and sometimes password-manager integrations. A Robot that performs recurring web work may need a persistent browser profile in its home so that every task does not begin with a new login. Yet that persistence can turn a compromised web page into a durable security problem if isolation and policy are weak.

The platform should therefore separate the usefulness of persistent browser state from the authority represented by that state. A Robot may have a browser profile for a procurement portal without having access to the organization's general single sign-on session. It may be allowed to read a dashboard but not submit payments. It may keep cookies but require a person for multifactor authentication. It may be restricted to a small set of domains or forced through network policy that blocks unapproved destinations. The goal is not to make browser automation impossible. The goal is to make the browser session a governed capability rather than an opaque box.



One Robot can use several modes when its configuration permits.

Figure 2. Three practical modes of Robot work. Browser use is separated from generic computer use because persistent web identity creates distinct security and usability concerns.

Figure 2 presents the three practical modes side by side. Background work emphasizes task folders and asynchronous completion. Computer use emphasizes a live application session with optional takeover. Browser use adds persistent web identity and web-specific controls. A single Robot may support all three, but an organization can also create specialized Robots whose permissions reflect only one mode.

The human takeover path should be deliberately simple. When a Robot pauses on a login challenge, the user should not need to create a new meeting, open a separate remote desktop product and reconstruct the session. The control should be available where the work is already visible. The user takes over, completes the sensitive step, and returns control. The Robot then continues with the updated screen state. This continuity is one of the places where integrating the runtime with the collaboration interface can create disproportionate value.

There is also a psychological benefit. People are more willing to delegate when they can verify. An employee learning to trust a new colleague often watches the first few tasks closely, then reduces supervision as performance becomes predictable. The analogy is imperfect, but the operational pattern is useful. A team can initially run a browser Robot in closely supervised mode. After observing repeated correct behavior on a narrow workflow, the team can allow more background execution. Autonomy becomes a policy that is earned through evidence rather than a marketing setting labelled “agentic.”

Guide the work while it is running

The most naïve vision of autonomous agents imagines a clean transfer of responsibility: the human states a goal, the agent disappears into a black box and the correct result emerges. Real work is rarely so cooperative. Requirements are incomplete, software behaves unexpectedly, websites change, credentials expire, sources disagree and organizations contain rules that cannot be inferred from public text. A useful system must therefore make interruption and clarification normal.

AssistOS can frame human intervention as a set of interaction rights. The user can observe what a Robot is doing. The user can send a new instruction while the task is running. The user can pause or stop execution. The user can take over a computer or browser session. The user can approve or reject a proposed action. The user can change the task scope. None of these mechanisms should require destroying the Robot’s environment or abandoning the task history.

The same principle applies inside WebMeet. A Robot participating in a meeting should not be treated as a magical silent recorder that later produces a summary. It can be a named participant with an explicit role. A human can address it through the meeting chat, ask it to change a diagram, ask it to retrieve a source or tell it to stop a background action initiated earlier. The other participants can see that the Robot is working. If it shares a screen, they can see the process. If it changes shared state, the action should be attributable. The meeting becomes an operational room rather than a transcript factory.

This model also creates a healthier answer to the question “how autonomous is it?” Autonomy is not one number. A Robot may be allowed to read files without approval but require approval to overwrite them. It may browse approved websites autonomously but require a person before submitting a form. It may run tests in a sandbox but be prohibited from reaching production. It may propose email messages but never send them without confirmation. These policies can be different for different roles and organizations.

The user experience should express these boundaries in ordinary language. A permission dialog that lists twenty Linux mounts and seven network capabilities is technically precise but practically useless to most people. The system should say what the Robot wants to do: “Read the Project Delta folder, create files in Results, run the local test command and access docs.example.com.” Administrators can inspect the technical mapping. Ordinary users should see the meaning.

Human intervention also provides a route for organizational learning. When a Robot repeatedly stops at the same point, that pattern is evidence. Perhaps the instruction is unclear. Perhaps a Ploinky Agent needs a new tool. Perhaps a credential should be delegated differently. Perhaps the workflow should remain human. A platform that records these interruptions can help teams distinguish automation opportunities from automation fantasies.

The deeper design claim is therefore modest but consequential. The future of AI work is not a choice between human work and autonomous software. It is an interface problem: how to move responsibility back and forth between people and software at the granularity demanded by real tasks. AssistOS attempts to make that handoff visible, persistent and governable. Everything else in the architecture exists to support that experience.

Chapter 2 Organizing the shared workplace

The project connects instructions, files, running tasks and collaboration. Explorer, WebChat and WebMeet provide different ways to work with that shared context.

Find projects and artifacts in Explorer

Once the Robot concept is clear, the next question is where the work lives. AssistOS answers with Explorer, the browser-based workspace that users see. Explorer is not intended to replace the operating system kernel beneath the machine. It is a control surface over projects, files, tools, Robots and collaboration. In the same way that a browser became a primary environment for networked work without replacing Windows, Linux or macOS, Explorer can become the environment in which agentic work feels coherent without pretending to be a new low-level operating system.

The design begins with objects rather than conversations. A project contains folders, documents, code, data, images, recordings, specifications and other artifacts. These objects have names and locations that people can understand. A user can open a file, inspect its history, see which Robot worked on it and launch a relevant tool. Conversation remains important, but it is attached to work rather than acting as the only place work can exist.

This is the sense in which AssistOS turns the operating-system interface into an IDE for any kind of work. A financial analyst should be able to inspect a proposed spreadsheet change, rerun the calculation and restore the previous accepted version. A researcher should be able to reproduce a transformation from its sources. A writer should be able to compare revisions before accepting them. Explorer brings these practices into the everyday workspace; the Copilot makes them usable through ordinary requests rather than requiring everyone to learn developer commands.

This distinction matters because chat interfaces have weak spatial memory. A person may upload a document, receive a useful answer and later struggle to reconstruct which version was used or where the derived file went. Explorer restores a mental geography. The user sees the project and the artifacts that persist across model changes and agent upgrades. If an Agent is replaced, the project remains. If the model provider changes, the project remains. The durable asset is the work environment.

Explorer also becomes the natural place to display Robot state. A project can show that the Build Engineer is working in a task folder, the Web Operator is waiting for user input and the Research Robot completed a report. The user can inspect results without asking the Copilot to narrate the whole system. This is an important interface principle: AI should reduce the need for command syntax, but it should not force every fact about the environment to be discovered through language.

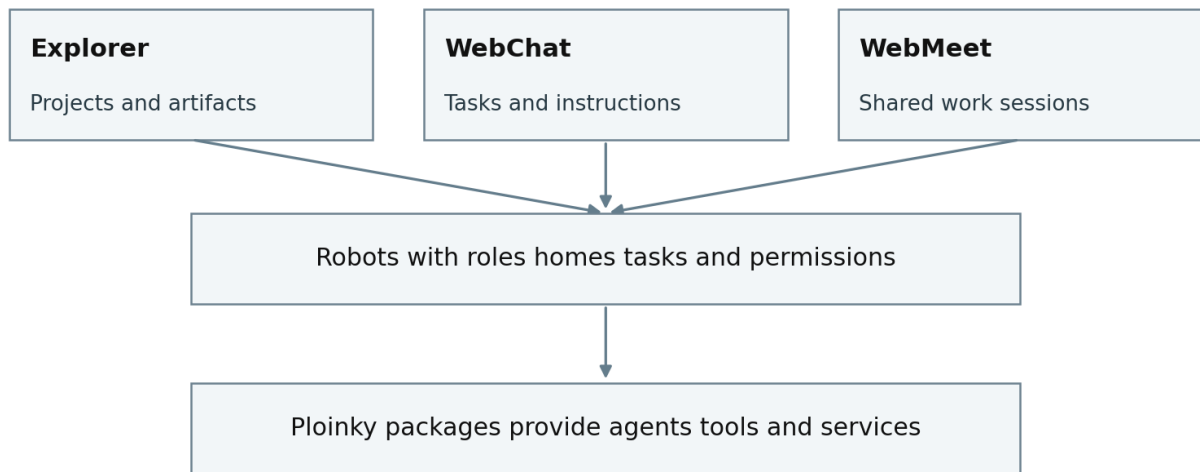


Figure 3. The AssistOS workplace. Explorer, WebChat and WebMeet are the visible surfaces; Robots preserve role and continuity; Ploinky deploys the capabilities underneath.

Figure 3 shows the workspace as a set of visible layers rather than a deep technical stack. Explorer contains the objects the user cares about. WebChat dispatches and coordinates tasks. WebMeet creates a shared room for human and Robot collaboration. Robots provide persistent worker identities. Ploinky deploys the capabilities underneath them. The host, local server or cloud infrastructure sits below the user’s normal attention.

Plugin architecture makes this workspace useful beyond one domain. A research team may need a paper viewer, bibliography tool and notebook. A software team may need an editor, terminal, repository browser and test dashboard. A creative team may need an image board and timeline. AssistOS should not hard-code one universal interface and then force every profession into it. Explorer provides the common frame; plugins provide specialized work surfaces.

There is a useful constraint here. Plugin flexibility can recreate the complexity the platform is supposed to remove. A good Explorer does not present fifty tool names on the first screen. It presents the project and the relevant work. Plugins should be discoverable in context. A spreadsheet-like artifact should open with a suitable editor. A running Robot task should open with its progress and working folder. A WebMeet room should appear when the user is collaborating. The system should feel integrated because the project supplies context.

For an enterprise, Explorer is also the point at which governance can become legible. The same object can show who owns a project, which Robots are allowed to enter it, what data classification applies and whether external services are permitted. Security is stronger when the policy can be seen where the work happens. An administrator may define the formal rule elsewhere, but the user benefits from seeing its practical consequence in the workspace.

Explorer therefore carries a large part of the product risk. The underlying runtime can be sophisticated and still fail commercially if the surface feels like an infrastructure console. The interface must hide incidental complexity while preserving meaningful control. A researcher should not manage container ports. A manager should not inspect an MCP transport trace to understand why a Robot cannot access a file. The platform should translate technical conditions into user-level explanations and offer a route to deeper detail for those who need it.

Assign and continue tasks through WebChat

WebChat is the conversational surface of the AssistOS Copilot. A user can express an objective, assign it to a Robot and continue a task when it needs input. Ploinky provides the underlying browser interface to configured capabilities. The full AssistOS dispatch experience also requires task routing, workspace context and integration with the installed Robots. [P9]

Suppose a user types, “Ask the Research Robot to review everything in the Contracts folder and identify clauses that changed from last year.” The Copilot should resolve several pieces of context. Which project is active? Which Robot has the relevant capability? Is the Contracts folder allowed for that Robot? Should the work run as a background task? Where should the result be stored? If the request is ambiguous, the Copilot asks the smallest useful question. Once the task is defined, it becomes a visible task rather than remaining a hidden internal tool call.

This distinction between conversation and task is essential. A conversation is an interaction history. A task is a unit of accountable work. It has status, scope, artifacts, limits and an owner. The Copilot can create tasks from conversation, but the user should be able to inspect them independently. That allows the conversation to remain readable even when several Robots work in parallel.

WebChat also provides continuity when a task needs input. A Robot can report, “I reached the customer portal and need a login confirmation,” and the Copilot can present the interaction in context. If the user sends a new instruction, it is routed to the relevant task rather than interpreted as a completely new request. This is especially important once multiple Robots are active. The Copilot becomes a traffic controller for human intent.

The interface should resist the temptation to simulate an office through personality alone. A charming Robot name can help users remember roles, but operational trust comes from evidence. WebChat should show what task is running, which folder it uses, what it produced and what it is waiting for. Personality can make communication pleasant. It should not obscure state.

Another important function is capability discovery. A user may not know that a Ploinky Agent exists for a particular analysis task. The Copilot can suggest it in terms of outcomes: “There is an installed data-analysis capability that can inspect these CSV files locally.” The user does not need to understand the packaging format. If installation is required, the Copilot can explain the permission and resource implications before proceeding.

For companies, WebChat can also reflect organizational policy. A user might ask a Robot to send a customer message, while policy allows drafting but not sending. The Copilot can explain that boundary and create a draft task instead. A user might request an external model for confidential material, while policy requires local processing. The system can route the work locally or request an exception. Good governance becomes less frustrating when the interface offers a useful alternative rather than a generic denial.

The dispatch-desk metaphor also clarifies what WebChat should not become. It should not be the only place to edit a document, inspect a code diff, watch a browser or control a meeting. Specialized interfaces remain valuable. Conversation connects them. This preserves one of the strongest insights from the AssistOS vision: language is an orchestration layer, not a reason to abandon every visual interface humans have learned to use.

Work together in WebMeet

Most AI meeting products begin from transcription and summarization. Those are useful functions, but they leave the structure of work unchanged. Humans talk, the meeting ends, and someone still

has to convert decisions into systems. WebMeet explores a different possibility: the meeting itself can be an interface to the working environment, with Robots participating as visible software members of the RoboTeam.

A WebMeet session can include ordinary video conferencing and screen sharing among people. The important extension is that Robots can also participate. Their presence does not require a synthetic face. A Robot can be represented by its name, role, current task and shared work surface. If it is operating a browser or desktop, that screen can be shared. If it is modifying a collaborative blackboard, the change can appear in real time. If a human sends a command through meeting chat, the Robot can translate that command into a task or an action within its permissions.

The same design applies to office documents. An editor should expose a shared artifact and controlled operations that both people and Robots can use. During a meeting, a Robot might propose a new paragraph or recalculate a table while participants inspect the change in the document itself. Review status, attribution and version history should survive after the call ends. Robot participation is therefore part of the document and meeting model from the outset, rather than an unrelated transcript attached afterward.

Imagine a product-design meeting. Participants share a diagram and ask the Robot through meeting chat to add the new authentication service between the gateway and data layer, then connect it to the audit component. The Robot is being asked to modify the shared object. The team sees the proposed change and can correct it immediately. The exact command or interaction depends on the installed meeting and diagram integrations.

A second pattern is the live evidence assistant. During a research meeting, a participant questions whether a claim is supported by the cited paper. The Research Robot can retrieve the source from the project, locate the relevant passage and place the evidence beside the discussion. The value is not conversational fluency. The value is shortening the path from question to inspectable artifact.

A third pattern is supervised execution. A Robot may already be running a background task when the meeting begins. The team can inspect its status, redirect it or stop it from within the meeting. If it encounters a graphical interface, a participant can temporarily take control. The boundary between synchronous collaboration and asynchronous work becomes porous in a useful way.

Table 4. A Robot can participate in WebMeet in different ways

Term or aspect	Meaning or practical use
Evidence assistant	Retrieves approved project material or external sources and places evidence beside the discussion.
Shared-state editor	Changes a diagram, blackboard, document or other collaborative object under visible attribution.
Live operator	Shares a browser or desktop session that humans can watch, pause or temporarily control.
Background worker	Runs a task initiated from the meeting and reports status or results back into the collaboration context.

The table distinguishes several forms of Robot presence because not every meeting needs the same level of automation. A Robot can be present only as a source retriever. It can operate a shared object. It can run a bounded task in the background. It can share a live computer session. WebMeet should make the current role visible so participants know what authority the Robot has.

This visibility is also important for social reasons. People should not have to guess whether an AI system is listening, recording, acting or storing information. A trustworthy collaboration

environment should show which Robots are present and what they can access. If a Robot joins because one participant invited it, organizational policy may still restrict which project artifacts it can see. Presence is not equivalent to universal access.

WebMeet therefore connects two ideas that are usually developed separately: collaboration software and agent orchestration. Collaboration products understand rooms, participants and shared state. Agent platforms understand tools, tasks and execution. AssistOS can create value by treating these as one operating context. A decision can become a task without re-entering it elsewhere. A Robot result can become part of the shared object while the discussion is still active. Human corrections can become new instructions immediately.

The larger promise is not “AI in meetings.” It is reducing the distance between conversation and outcome. Organizations lose enormous time when decisions have to be manually copied into tickets, documents, diagrams, spreadsheets and emails after every meeting. A platform where humans and Robots share the same project can make the decision and the resulting action part of one history.

Preserve artifacts and task histories

If AssistOS succeeds, the most valuable thing it accumulates is not chat history. It is the structured memory of work. Projects contain artifacts. Tasks explain how those artifacts changed. Robot identities and capability versions explain who or what performed the work. Approvals explain why a consequential action was accepted. This history can survive model replacement because it belongs to the project rather than to a single model provider.

This is particularly important in long-running work. Consider a scientific report revised over six months. A sequence of different models may be used as costs and capabilities change. One Robot may be reconfigured with a new research Agent. Another may be retired. If the only memory is embedded in proprietary conversation transcripts, the project becomes fragile. If sources, drafts, decisions and task outputs are stored as ordinary project artifacts with provenance, the organization retains continuity.

Provenance does not require that every keystroke become a cryptographic event. Excessive logging can be as unusable as no logging. The platform should preserve the level of history needed to answer practical questions. Which input file did the Robot read? Which version did it produce? Which task created the output? Which human approved publication? Which capability and model were used for a significant step? The appropriate detail depends on risk and domain.

The artifact model also creates a better interface for review. When a Robot edits a document, the user should be able to inspect the new version against the old one. When a Robot changes code, the user should see a diff and tests. When a Robot produces a dataset, the user should know its source files and transformation script. The review surface should fit the artifact rather than forcing every result into a paragraph of explanation.

Task history connects these artifacts across time. A task can point to its inputs, outputs, progress events and approvals. A later task can reuse the result while preserving lineage. This is where the design of reusable procedures becomes relevant. A good task description is a lightweight specification. Over time, frequently repeated tasks can become more formal, reusable processes. The system can move gradually from ad hoc delegation to reproducible workflows without requiring the user to design a workflow graph on day one.

The history also supports learning about Robots. A team can examine which tasks a Robot completes reliably, where it requests intervention and where it fails. That evidence can inform policy. A Robot that repeatedly performs a narrow operation safely can receive more autonomy. A

Robot that frequently misunderstands a category of task can be restricted or reconfigured. Performance becomes empirical rather than anecdotal.

For regulated or high-stakes environments, the same history can support audit, but AssistOS should avoid making universal compliance claims. Audit requirements differ across sectors and jurisdictions. The platform can provide technical ingredients such as identity, task records, versioning, approvals and policy enforcement. Whether those ingredients satisfy a particular regulatory obligation requires domain-specific configuration and validation.

The broader strategic point is that project memory is a durable asset. Models will improve. Agent frameworks will change. The company that owns a coherent, portable history of its work is less dependent on any one AI supplier. AssistOS can create switching freedom without sacrificing continuity, which is a more attractive form of retention than trapping users inside an opaque memory store.

Keep approved work distinct from proposals and derived results

A project becomes easier to manage when the workspace distinguishes an original, a proposed revision and an accepted version. These are relationships in the work, not merely different filenames. In the supplier comparison, a draft report may be useful while still awaiting a check of delivery dates. Another Robot should not treat that draft as an approved purchasing decision simply because it is the most recently modified file.

The same distinction applies to a book. An author may ask for a more accessible opening while preserving the accepted chapter. The Writing Robot creates a candidate revision. An editor can compare the versions, accept selected changes and retain an explanation for rejecting others. If the team later generates an audio script or presentation, it should specify which accepted manuscript version those derivatives use. Otherwise, several individually plausible artifacts can express different editions of the same work.

Version control and comparisons can also help people who never use a programming tool. The person can ask to preserve the current version or compare two alternatives in ordinary language. An editor, a comparison view and explicit version relationships make the request concrete. The implementation may use familiar files and versioning tools underneath. [P6, Chapter 4]

Derived results also need dependency information. Suppose a proposal's delivery date changes after the comparison report has been accepted. The source change should make the affected comparison row, recommendation and presentation identifiable for review. It need not invalidate unrelated price extraction. A useful workspace can show that a result needs rechecking because one of its inputs changed. An automatic system would need reliable dependency tracking to determine precisely what to rerun; until then, the Robot can propose the affected items and a human can confirm them.

Project memory should preserve decisions as well as outputs. An accepted decision record can state that Supplier B was excluded because its service region does not cover the required location. A later Robot can use that record while checking whether the condition still applies. The record should point to its source and date, so an old decision does not become a timeless fact.

For a new colleague, this is more useful than reading every conversation. They can open the current artifact, inspect the decision that shaped it and follow the evidence when necessary. For the original author, it reduces the cost of returning to a project after an interruption. These are practical reasons to preserve provenance before introducing a more abstract knowledge graph.

Adapt the workplace with plugins

A general workplace platform faces a paradox. If it supports only a few built-in tools, it is too narrow. If it exposes every possible integration as a permanent interface element, it becomes an enterprise control panel that ordinary users cannot navigate. Explorer's plugin model is the proposed answer. The stable core provides projects, identity, permissions, Robot state and routing. Plugins provide specialized work surfaces.

The simplest plugin may be a viewer or editor for a file type. A more complex plugin may embed a notebook, a diagram editor, an IDE, a data dashboard or a domain-specific application. WebMeet itself can be understood as a high-value collaboration plugin. The user remains inside the AssistOS context while specialized functionality appears when needed.

A plugin can also be coupled to Ploinky Agents. The visual surface may be one half of the capability while the runtime service is the other. A scientific visualization plugin might call an analysis engine running in a container. An engineering plugin might interact with a coding Agent and repository tools. A document-review plugin might expose a side-by-side comparison while a Ploinky Agent performs semantic analysis. This separation allows the web interface to remain relatively lightweight while complex dependencies stay isolated below.

The visual interface and the Robot's tool interface should refer to the same objects. A person sees a paragraph and proposed revision; the Robot can receive an operation to propose a change to that paragraph. A spreadsheet can expose a defined range and a calculation operation while the person sees the corresponding cells. Where these structured operations exist, they can be easier to test than repeated screen clicks. Browser and computer use remain valuable for applications that lack such interfaces. Both routes should leave the effect visible in the project. [P6, Chapter 9]

The plugin contract should preserve the platform's security model. A plugin should not automatically inherit access to every file or every Robot simply because it renders inside Explorer. The platform can grant capabilities explicitly and expose project-scoped APIs. This is a familiar lesson from browser and mobile ecosystems: extensibility without capability boundaries becomes an attack surface.

For developers, the plugin model can create a distribution path. A specialized tool builder does not need to reproduce user management, collaboration, agent deployment and project navigation. They can focus on the domain-specific value. For AssistOS, third-party plugins expand the useful surface without requiring the core team to build every application. A marketplace can extend this distribution path after the tools have demonstrated their usefulness.

The user should not experience plugins as an app store full of experiments. Early adoption requires curation. A small number of excellent, understandable plugins is more valuable than thousands of poorly described integrations. The platform should make clear what a plugin does, what it accesses, what runtime components it installs and how it can be removed.

This emphasis on removal is important. An open ecosystem should preserve the right to leave. Projects should remain exportable. Plugins should use interoperable formats when possible. Robot capabilities should be replaceable. AssistOS can become a deeply integrated workspace without turning continuity into captivity. The architecture is strongest when users stay because the environment organizes their work better, not because their work cannot survive elsewhere.

A mature plugin system should also separate interface trust from runtime trust. A plugin rendered inside Explorer may be visually simple while relying on a Ploinky Agent that executes substantial code. The user should be able to understand that relationship. Installing a diagram plugin that only edits local JSON is different from installing a plugin that launches a container with network access

and a browser profile. AssistOS can present one coherent installation flow while still distinguishing the permissions and runtime consequences behind the interface.

Configuration should follow the same layering. Individual users need role-level choices such as which Robot may use the plugin and which project folders it can read. Teams need shared defaults. Administrators need organization-wide restrictions and approved versions. Developers need the full manifest and diagnostic information. Trying to expose all four views in one settings panel would recreate the complexity that Ploinky is meant to hide. Explorer can instead reveal detail progressively, beginning with what the capability does and expanding into how it is implemented only when the reader or administrator asks.

This progressive disclosure is especially important for AI features because the apparent simplicity of language can conceal substantial authority. A button labelled “Summarize” may trigger only a read-only model call, while a button labelled “Prepare release” may launch a coding Agent with shell access, create files, call remote services and open a browser. The interface should not make those operations look equivalent merely because both can be started with one sentence. Good product design communicates consequence without forcing the user to become a systems engineer.

Plugins also provide a path for organizational specialization. A company can build a small internal plugin that exposes a domain-specific workflow while reusing the common Robot, task and policy model. A laboratory may integrate an instrument dashboard. A legal team may integrate its document comparison viewer. A software company may integrate a deployment console. These surfaces can remain private while the underlying AssistOS concepts stay the same. This reduces the pressure to turn every organizational exception into a feature of the public core.

Chapter 3 Applying AssistOS to useful work

Choose a workflow whose inputs and outputs are clear. These examples show how the Robot model applies to software, research, office work and personal collections, then explain how to evaluate a first pilot.

Develop and review software

Software engineering is a strong first domain for AssistOS because coding agents already demonstrate the core pattern of delegated digital work. A modern coding agent can inspect a repository, reason about a task, modify files, run commands and evaluate tests. The work is naturally artifact-centered, version control provides recovery, and technical users understand that an agent can be productive without being infallible. The remaining problem is operational: how to run multiple such workers safely, persist their roles, supervise them, connect them to projects and avoid turning every experiment into a special infrastructure setup.

A developer using AssistOS might create several Robots with distinct responsibilities. The Implementation Robot works on bounded feature tasks. The Reviewer Robot reads changes but does not write to the main branch. The Release Robot can prepare release notes and execute approved test procedures. These Robots may all use the same underlying coding Agent, or each may use a different one. The distinction in role is valuable because permissions and expectations differ even when the cognitive engine is identical.

A task can begin in WebChat: “Implement the pagination behavior defined in specs/search.md, work in a separate worktree, run the unit tests and stop if the database migration changes.” The Copilot binds that instruction to the project and the Implementation Robot. The Robot launches its coding Agent in the task workspace, reports progress and returns a patch. The developer can inspect the diff in Explorer, ask for a correction or merge the result manually.

The same pattern supports parallelism. Instead of one interactive coding session, the developer can give three independent tasks to three Robot instances or to one RoboTeam. One investigates a test failure. One updates documentation. One prototypes a migration in a disposable environment. The human moves into a supervisory role, deciding which tasks are sufficiently independent and reviewing results. This is not automatically faster; poor task decomposition can create merge conflicts and duplicated work. AssistOS can add value by making the parallel work visible and giving each task an explicit folder and history.

Table 5. Software work maps cleanly to reviewable Robot outputs

Term or aspect	Meaning or practical use
Feature implementation	Patch or worktree, tests and a short completion report.
Code review	Findings tied to files and lines, without write access when the role is review-only.
Test investigation	Reproduction steps, logs, suspected cause and a proposed fix in a disposable branch.
Release preparation	Version changes, release notes, build results and explicitly approved publication steps.
UI reproduction	A visible browser or desktop session plus screenshots, logs or test artifacts.

The table shows why software engineering fits the model well. Most useful outputs are files, tests or structured reports. Sandboxes can isolate package installation. Version control makes many changes reversible. Permissions can be narrow. The domain therefore provides a demanding but manageable environment in which the Robot architecture can mature.

Computer use still matters even in software engineering. A Robot may need to operate a local graphical test application, inspect a browser-based admin console or reproduce a user-interface bug. The developer can watch the session and intervene. A coding Agent alone may not provide this capability; a Ploinky Agent can combine the coding agent with browser or desktop tools in the Robot's environment.

There is also a role for deterministic tools. Static analyzers, compilers, linters, test frameworks and security scanners should remain first-class. A language model should not replace a unit test with a confident statement that the code "looks correct." The Robot can orchestrate these tools and interpret their results. AssistOS is strongest when it combines ordinary software engineering discipline with AI rather than treating AI as an exemption from discipline.

For a company, the economic argument is not simply that a Robot writes code faster. The larger benefit can be reducing context switching and queueing. Small maintenance tasks that would otherwise wait for an engineer can be delegated and reviewed later. Documentation can be updated in parallel with implementation. Repetitive repository analysis can become a background service. Senior engineers can spend more time on architecture, ambiguous requirements and consequential review.

The main risk is over-delegation. Coding agents can generate large volumes of plausible code that creates future maintenance cost. AssistOS should therefore make quality evidence visible: tests, diffs, static analysis and review status. The virtual worker should increase the amount of reviewable engineering work completed, not merely the amount of code produced.

Investigate questions and preserve research evidence

Research is another natural domain because it combines heterogeneous artifacts, uncertain goals, external sources and repeated cycles of critique. A researcher does not simply ask questions. They collect papers, maintain notes, compare hypotheses, analyze data, design experiments, write reports and defend claims. A chat assistant can help at each step, but continuity across the steps is often weak. AssistOS can organize the work around a project and a team of specialized Robots.

A Literature Robot may be responsible for source discovery and bibliographic organization. A Critic Robot may challenge claims and look for contradictory evidence. A Data Robot may run notebooks or scripts in a controlled environment. A Reproducibility Robot may attempt to rerun code and record dependency problems. A Writing Robot may transform accepted notes into a readable draft while preserving source links. These roles can be configured independently and can use different Ploinky Agents.

The workflow becomes more powerful when outputs are artifacts. The Literature Robot writes a source manifest and annotated notes. The Critic Robot writes objections tied to specific claims. The Data Robot writes code and generated tables. The Writing Robot uses only the accepted material and produces a draft in the project. The human can inspect the lineage rather than relying on a single long conversation whose intermediate reasoning is difficult to audit.

A research task can begin informally and gradually become a reusable procedure. A literature review may acquire explicit source rules. A writing procedure may require factual claims to point to

evidence. A data-analysis procedure may require scripts to be saved with their outputs. Chapter 4 explains how these conditions can be recorded, checked and improved across runs.

Table 6. Separate research roles reduce the temptation to make one model judge itself

Term or aspect	Meaning or practical use
Literature Robot	Finds and organizes sources under explicit inclusion rules.
Critic Robot	Searches for counter-evidence, contradictions and unsupported claims.
Data Robot	Runs scripts, notebooks or deterministic analysis tools in a controlled environment.
Reproducibility Robot	Attempts to rerun methods and records missing dependencies or irreproducible steps.
Writing Robot	Transforms accepted material into readable prose while preserving links to source artifacts.

The table demonstrates that the “RoboTeam” concept is especially useful when one model should not play every epistemic role. A system that both generates a hypothesis and evaluates it in the same conversational context can reinforce its own errors. Separate roles do not magically remove bias, but they create opportunities for adversarial checking, independent prompts, alternative models and deterministic validation.

Axiologic Research also investigates ways to make AI-assisted knowledge work more inspectable through SOP Lang circuits and S*LLM, the Small, Symbolic, Specialised Language Model direction. These are research approaches to narrower language semantics, explicit constraints and verification alongside LLMs. They should be evaluated on defined tasks rather than treated as universal truth engines. The process and research distinctions are developed in the final chapter.

WebMeet adds another dimension. A research group can discuss a hypothesis while a Robot retrieves evidence, updates a shared diagram or starts an analysis task. The meeting history can connect the decision to the resulting artifact. A collaborator who joins later can inspect what was accepted and why. This is closer to an executable laboratory notebook than to a meeting transcript.

For organizations, the attraction is not limited to academic science. Legal review, policy analysis, technical due diligence, standards work and strategic research share the same structure: many documents, uncertain claims, need for evidence and repeated revision. AssistOS can provide a general environment while domain-specific Ploinky Agents and plugins supply the specialized tools.

The danger is epistemic automation without verification. A Robot that produces a polished report can make weak evidence look authoritative. The interface should therefore privilege sources, uncertainty and reproducible artifacts. A successful knowledge-work Robot should make it easier to inspect why a conclusion exists, not merely easier to generate conclusions.

Operate websites and administrative applications

A large portion of office work remains trapped in websites. Employees copy values from one portal to another, download statements, update customer records, submit forms, reconcile dashboards and check status pages. Traditional robotic process automation addressed some of this work by scripting interfaces, but scripts can be brittle when layouts change. AI-based browser Agents promise more adaptability because they can interpret screens and language. AssistOS provides a place to deploy such Agents with persistent identity and human supervision.

The easiest use cases are read-oriented. A Browser Robot can log into an approved portal, collect a report, download invoices, check application status or compare values across systems. These tasks are attractive because mistakes are often reversible and the Robot can return artifacts for review. A persistent browser profile reduces repeated authentication friction.

Write-oriented tasks require more caution. Updating a CRM field is relatively low risk. Submitting a regulatory form is higher risk. Sending a payment is much higher. The platform should allow organizations to create different approval boundaries. A Robot may fill a form and stop before submission. It may draft a message and wait for approval. It may execute low-risk changes automatically but require a human for irreversible actions.

Computer use extends this beyond the browser. A company may have a desktop accounting program, a VPN-only administration console or a specialized client. If the application can run inside or be reached from the Robot’s environment, the Robot can operate it while the human watches. This is particularly valuable for legacy systems that will never gain modern agent APIs.

Table 7. Browser automation should be classified by consequence

Term or aspect	Meaning or practical use
Read-only status	Usually a strong automation candidate when access is scoped and outputs are reviewable.
Data entry	Often automatable with validation and a clear undo path.
External submission	Frequently benefits from a human confirmation before the final action.
Customer communication	Drafting can be automated more aggressively than sending under a person or company identity.
Financial or irreversible action	Requires strong policy, explicit authorization and domain-specific safeguards.

The table divides browser work by consequence rather than by technical difficulty. This is a better basis for automation policy. A task may be easy for an Agent but still inappropriate for full autonomy because the business consequence is large. Conversely, a technically messy read-only workflow may be safe to automate aggressively because the Robot cannot change external state.

Browser operations also expose the importance of session continuity. If a user has to log in manually every time, automation loses much of its value. If the Robot keeps a broad browser profile containing every company session, the security boundary becomes unacceptable. Dedicated Robot profiles provide a compromise. The Procurement Robot can have the procurement portal session without also inheriting the user’s personal email session.

The human takeover model is critical. A website may request an MFA code, display a new terms-of-service page or show a CAPTCHA. The Robot should not improvise around such barriers. It can pause, surface the live session and let the user complete the human step. Afterward, the Robot resumes. This makes mixed-initiative automation a first-class workflow rather than an awkward exception.

Administrative work also demonstrates why AssistOS should not promise that AI eliminates people. Many workflows encode organizational authority, legal accountability and tacit judgment. The better objective is to remove repetitive interaction while preserving meaningful human decisions. A finance employee may spend less time downloading and copying data and more time investigating anomalies. A project coordinator may spend less time updating tickets and more time resolving dependencies.

The practical measure of success is cycle-time reduction without unacceptable error. A company should be able to compare a workflow before and after Robot assistance: how many human minutes are required, how often intervention occurs, how often the Robot makes a correction-worthy mistake and what the infrastructure costs. AssistOS should make these metrics available because they turn “agentic automation” into an operational investment decision.

Create documents and shared creative work

Not all valuable work is a transaction or code change. Teams spend much of their time creating shared understanding: proposals, specifications, reports, diagrams, designs, presentations and plans. These artifacts evolve through conversation and revision. AI can accelerate them, but current workflows often separate the AI conversation from the collaborative object. AssistOS can close that gap.

A team drafting a technical specification can open the document in Explorer and discuss it in WebMeet. A Specification Robot can be asked to restructure a section, identify contradictions or add missing acceptance criteria. The change appears as a proposed version rather than replacing the original silently. A Diagram Robot can modify a shared blackboard based on meeting chat. A Research Robot can attach sources to contested claims. The humans remain responsible for the artifact while Robots perform targeted work around it.

Creative work benefits from the same pattern. An editor can ask a Writing Robot to generate alternative passages while a Fact Robot checks historical references. A design team can ask a Robot to organize assets, prepare captions or create derivative formats. A video-production workflow can use specialized Ploinky Agents for transcription, audio processing or rendering. The important point is that the system coordinates many kinds of capability under one project rather than claiming that text generation alone is the creative process.

The shared-object model also reduces ambiguity about what is final. In ordinary chat-based collaboration, a person may paste a paragraph into chat, receive three alternatives and later forget which one entered the document. Explorer can keep the document as the authoritative object. Robot suggestions are versions or proposals attached to it. WebMeet conversations refer to the same object. This preserves a clear center of gravity.

Table 8. Useful verbs for Robot participation in shared work

Term or aspect	Meaning or practical use
Propose	Create a candidate change while preserving the authoritative original.
Critique	Identify weaknesses, missing evidence or inconsistencies without changing the object.
Transform	Convert an artifact into a new form such as summary, translation, presentation or cleaned dataset.
Operate	Modify a shared tool, diagram, browser or application under visible supervision.
Retrieve	Bring relevant project context or external evidence into the current collaboration.

The table distinguishes common patterns such as propose, critique, transform and operate. These verbs are more useful than generic claims that a Robot “helps.” Teams can decide which kinds of work a Robot is allowed to perform on which artifacts. A Writing Robot may transform prose but

not approve publication. A Diagram Robot may edit the blackboard but not delete source documents. A Meeting Robot may create tasks but not close them as completed.

The same architecture can support education and training. An instructor can work with students and Robots on a shared artifact, observe which Robot actions were requested and review the resulting changes. A mentoring Robot can suggest resources while the teacher retains authority. Again, the value comes from transparent collaboration rather than replacing the human relationship.

AssistOS should be careful with generated media and provenance. As image, audio and video generation become common, the project should record which assets were generated, which models or tools were used and which human approved the result when that information matters. This is not only a compliance concern. It helps teams manage versions and maintain editorial responsibility.

The larger lesson is that meetings, documents and creative tools should not be separate AI islands. If a Robot can move between them under the same identity and project context, the organization spends less effort re-explaining the task. The shared project becomes the memory. The Robot becomes a participant in the workflow rather than a chatbot consulted from outside it.

Use the same pattern for personal and professional collections

The Robot model should also work for a person with an ordinary collection of files. A household may have appliance manuals, warranty documents, invoices and renovation proposals scattered across folders. A consultant may have contracts, previous reports and client notes. In both cases the first useful task is to organize a selected collection and produce something that can be checked.

For a household archive, a first assignment might be to list the appliances, purchase dates and warranty periods found in the supplied documents. The Robot returns a table with a source reference for every extracted date and leaves unknown values empty. It can suggest duplicate files for review without deleting them. A second task might compare repair instructions from two manuals. The person chooses the collection; the Robot's useful memory consists of the index, corrections and accepted organization of those files. [P6, Strategic Appendix]

For a consultant, document translation provides another clear example. The request can specify the source files, target language, terminology to preserve and destination folder. The Robot produces translated Word documents and identifies passages whose meaning required judgment. The user checks terminology and layout in the resulting documents. A successful translation workflow must preserve tables, headings and relationships in the source, as well as the words. Changing the output language should not silently change the conversational language or the task's scope.

Local execution is useful when suitable tools and models can process the collection on the person's computer. A large scanned archive may exceed the available memory or require a specialized service. The meaningful choice is then which material may be processed where, with what expected result. A local web interface alone says nothing about whether the model runs locally; the task's actual processing route matters.

These workflows need little organizational machinery. One person can own the project, configure one Robot and review the results. The same person may later invite a colleague or add a second Robot for checking. Teams add shared responsibility around an already understandable activity, rather than requiring an enterprise deployment before anyone receives value.

The test of success is straightforward: can the person find an important document faster, verify an extracted fact, recover a previous version and continue the work on another day? Those outcomes

connect the larger AssistOS vision to daily use without depending on a future marketplace or research breakthrough.

Run a pilot and measure its value

The most convincing AssistOS deployment will not begin with a grand plan to transform the entire company. It will begin with one or two workflows where the value can be observed. The right first workflow has repetitive digital work, clear artifacts, moderate consequence and a human who understands the process well enough to supervise it. Software maintenance, research preparation, document transformation and read-oriented browser operations are often good candidates.

A useful pilot starts by measuring the current process. How many human hours does the task consume? Where does waiting occur? Which steps require judgment? What systems and credentials are involved? What does a good output look like? The Robot is then configured to take a bounded portion of the work. The team measures completion rate, intervention frequency, error rate, elapsed time and compute cost.

For an illustrative pilot, suppose twenty comparable tasks require thirty human minutes each under the existing process. That is ten hours. If Robot assistance reduces preparation, intervention, review and repair to fifteen minutes per task, the total becomes five hours. The apparent saving is five hours, before amortizing setup and maintenance. These are example numbers, not AssistOS performance results. A credible pilot records the actual values and includes failed tasks in its accounting.

A workflow with one deliberate review can be more useful than a workflow that operates unattended but creates costly repair. Human attention, error consequences and execution cost should be assessed together. The pilot should measure accepted work after corrections, rather than treating autonomous execution as the success criterion.

Table 9. A pilot should measure operational value, not just demo quality

Term or aspect	Meaning or practical use
Cycle time	Elapsed time from task assignment to reviewable result.
Human minutes	Active human attention required before, during and after the Robot task.
Intervention rate	How often the Robot requires clarification, takeover or approval.
Correction rate	How often a completed result needs substantive human repair.
Execution cost	Model, compute, storage and external service costs per useful task.
Reversibility	How easily errors can be undone; this should influence the allowed autonomy level.

The table turns the pilot into an engineering exercise. A buyer can evaluate whether the Robot actually reduces cycle time, whether interventions are becoming less frequent and whether errors remain recoverable. These measures also indicate when to stop. If a workflow requires constant human correction, the problem may not be mature enough for automation or the chosen Agent may be unsuitable.

There are several classes of work that should remain human-led even if AI is involved. Decisions with substantial legal, medical, financial or safety consequences may require qualified human responsibility. Ambiguous interpersonal situations can be harmed by automated communication. Strategic choices that define organizational values should not be outsourced merely because a model

can write a rationale. AssistOS can support analysis and preparation without pretending that the platform removes accountability.

The system should also resist automating dysfunction. If a company has a pointless approval process, building a Robot to click through it faster may preserve a bad process. Sometimes the better intervention is to redesign the workflow. Robots make hidden coordination costs visible, which creates an opportunity to simplify rather than only automate.

For individuals, adoption can be even simpler. A consultant may create one Research Robot and one Browser Robot. A writer may create a Fact Robot and an Editorial Robot. A software developer may start with one coding Robot whose only permission is a project directory. The product should be useful at this scale before enterprise features are required.

The long-term adoption path is gradual. A user first delegates a task. Then they reuse the Robot. Then they allow more autonomy in a narrow domain. Then a team shares the Robot configuration. Then the organization curates approved capabilities and policies. AssistOS becomes infrastructure because it repeatedly solves small operational problems, not because users are asked to believe in an abstract future first.

Chapter 4 Making delegated work repeatable and controllable

A useful first result becomes more valuable when the method can be reused, corrected and evaluated. The same work also needs explicit authority, visible execution and recovery when something goes wrong.

Turn a successful task into a reusable procedure

After a Robot has completed a useful task, the next question is how to repeat it without reconstructing the instructions. A reusable procedure records the objective, expected inputs, sequence of work and acceptance criteria. It can begin as a short document in the project. It does not require the user to draw an execution graph or learn a new programming language.

Return to the supplier comparison. The first run exposed two recurring issues: proposals use different price periods, and delivery dates are sometimes conditional. The consultant can improve the procedure by requiring both fields to preserve their original wording and to state any normalization assumption. These corrections become part of the next task brief. Reuse means carrying forward the method while binding it to the new proposals; it does not mean copying last month's facts into this month's report.

A procedure separates preparation from acceptance. The Robot may extract the proposal fields, compare them against the requirements, write a recommendation and prepare the Word report. The extraction can be checked against the source pages. Arithmetic can be checked by ordinary code. The recommendation can be reviewed for assumptions and tradeoffs. The final document can be inspected for missing sections and broken layout. Each stage has a result that another stage or a person can examine.

The procedure also defines the division between learned and symbolic processing. A model may interpret a supplier's wording and propose normalized fields. A schema checks the field types, ordinary code computes comparable totals, explicit rules flag missing conditions, and a person reviews unresolved interpretations. A smaller local model becomes useful when this surrounding structure reduces the reasoning it must perform. For acceptance conditions that must hold exactly, a fluent answer is insufficient: those conditions need an independent check. The check remains limited by the correctness of its rules and inputs.

Table 10. Repair follows the result that changed

Change during the comparison	Work to revisit
A source price was corrected	Its extracted value, normalized comparison and dependent recommendation
A delivery condition was clarified	The affected delivery comparison and any decision using it
A report page break was repaired	Layout and final document checks
The buying objective changed	The task specification and all stages affected by the new criteria

Change during the comparison	Work to revisit
Only the output model changed	Generation and acceptance checks using that model; reuse earlier stages only when their conditions remain valid

A process preserves enough state to explain which inputs, tools and decisions produced a result, and where a failed stage can be repaired. The practical benefit is continuity when work takes more than one attempt. [P6, Chapter 5]

Suppose the extraction tool misreads a scanned price. Correcting that value should trigger a review of the comparison and recommendation that depend on it. It should not require downloading unrelated documents again. Conversely, fixing a page break in the final report should not automatically rerun source discovery. Selective repair is justified by dependencies: preserve the stages whose inputs and acceptance conditions remain valid, then rerun or review the affected stages.

That dependency rule has limits. A new instruction to compare total ownership cost rather than purchase price changes the objective. It may invalidate the fields collected earlier and require additional evidence. A previously accepted extraction is also not reusable merely because its filename is unchanged; its content and extraction method may have changed. The project needs versioned inputs and an explicit record of what each stage used.

A run record should therefore identify the procedure version, input versions, capability and model choices where relevant, output artifacts, checks and review decisions. For probabilistic generation, reproducibility usually means that another run can be compared under preserved conditions, not that it must generate exactly the same prose. Deterministic arithmetic or a test result can have a stricter expectation when its environment and inputs are fixed.

The person using AssistOS should see the practical consequence: the Robot can explain what needs to be done again and retain the unaffected work. A more advanced process engine may automate this bookkeeping, but the workflow is useful before that automation exists. Saved procedures, explicit stage outputs and review records already establish a disciplined way to delegate.

Improve the procedure using failures and corrections

Repeated tasks produce evidence about the procedure. A team can discover that scanned tables cause most extraction errors, that an instruction leaves the delivery period ambiguous or that the final report takes longer to review than expected. These observations identify something to change. A growing task history alone does not demonstrate that the Robot is learning or becoming more reliable.

The improvement may be modest: a clearer task template, a deterministic calculation, a more suitable extraction tool or a check before writing the recommendation. A coding agent can help propose such changes and run them in an experimental environment. The accepted procedure remains stable while the candidate is evaluated. Changing instructions, changing a tool and training model weights are different interventions; the project should record which one occurred.

For example, collect representative supplier comparisons for which the expected fields have been reviewed. Include a scanned document, incompatible currency or period conventions, a missing value and a proposal with conditions expressed in footnotes. Reserve some cases for evaluation after the changes are made. Repeatedly tuning a procedure against every available example can hide overfitting to that collection.

The comparison should measure usable outcomes: correct field extraction, preservation of missing information, valid source references, review time and total execution cost. A candidate that writes more elegant prose but omits an important condition should fail acceptance. A candidate that saves model cost while doubling human repair time may also be worse. The team defines the tradeoff before choosing the next production version.

A change can then be adopted for the workflow it actually improved. A better supplier-comparison routine is not evidence that the Robot has become generally competent at accounting, legal interpretation or negotiation. When a new model or package version is introduced, the same evaluation cases help detect regressions. If quality falls, the team can return to the prior procedure while preserving the failed run for diagnosis.

Operational learning improves the system that selects, sequences, checks and repairs available capabilities. It is a useful development direction because it turns review into operational knowledge. Its credibility depends on measured comparisons and controlled changes, rather than an unobserved claim of self-improvement. [P6, Chapter 5]

Define the execution boundary

The more capable a Robot becomes, the more seriously its execution environment must be treated. A coding agent can run shell commands. A browser agent can act under an authenticated session. A data agent can read sensitive files. These are not hypothetical risks introduced by malicious software alone. A well-intentioned AI Agent can make destructive mistakes because it misunderstood a goal, followed hostile instructions embedded in content or simply encountered an unexpected system state.

Sandboxing separates a running program from resources it should not freely control. Containers, operating-system isolation and restricted runtime configurations can contribute. For a Robot, the relevant question is which files, credentials, network destinations and host resources remain accessible inside the chosen boundary.

AssistOS treats isolation at two related levels. A packaged capability needs a controlled installation and runtime boundary so its dependencies and processes do not freely modify the host. Each Robot also needs a private execution environment for its own writable home, temporary files, agent configuration and authorized credentials. Two Robots may share a read-only package image while using separate instances and mounts. This does not require nesting one container inside another; it requires separate, enforced state and authority boundaries. A home directory alone is not a sandbox.

Yet a sandbox should never be marketed as perfect containment. Containers share a host kernel. Configuration errors can expose files or network services. Privileged mounts can erase the value of isolation. Vulnerabilities can cross boundaries. Browser automation introduces its own attack surface. A secure design assumes that isolation can fail and layers controls accordingly.

The first layer is resource scope. A Robot should see only the files and devices required for its task. The second is identity scope. It should use its own credentials rather than inheriting the full authority of the human operator. The third is network scope. The Robot should not automatically reach every destination on the internet or internal network. The fourth is action policy. Some operations should require explicit approval even when technically reachable. The fifth is observability. Significant actions should leave enough evidence to investigate what happened.

Table 11. Layered boundaries for a software worker

Term or aspect	Meaning or practical use
File scope	Mount or expose only the project paths required for the task.
Identity scope	Use Robot-specific or task-scoped credentials rather than unrestricted human credentials.
Network scope	Restrict external and internal destinations where practical.
Action policy	Require approval for consequential operations even when the environment can technically perform them.
Observability	Record significant actions, task state and resource use so behavior can be investigated.
Recovery	Preserve versions, disposable sandboxes and rollback paths for reversible work.

The table is intentionally redundant. Security layers overlap because individual controls fail. If a coding Robot cannot see production credentials, a prompt injection inside the repository has less power. If network egress is restricted, a stolen token is harder to exfiltrate. If sensitive changes require approval, an agent mistake is less likely to become a production incident. If task history records what ran, recovery and investigation are easier.

Sandboxing also provides operational hygiene. An Agent can install dependencies without polluting the host. A failed experiment can be discarded. Two Robots can use conflicting tool versions. A new package can be tested in a disposable environment. These benefits are often as important to adoption as security because they reduce the fear that an AI tool will destabilize a developer's machine.

The platform should communicate these boundaries honestly. "Runs in a container" is not a sufficient security argument. An enterprise buyer will want to know which host resources are mounted, whether the process runs as root, how secrets are injected, what network routes are available, how images are updated and what logs exist. AssistOS should make these questions answerable without implying that one sandbox technology solves all of them.

The product opportunity is precisely here. As AI Agents become more capable, the market for isolation, governance and observability grows. Model vendors will provide their own environments. Cloud platforms will provide managed agent systems. An open and self-hostable AssistOS can differentiate by making boundaries portable across models and tools rather than tying governance to a single provider.

Set the scope of delegation

Traditional software security often speaks about least privilege: a user or process should receive only the permissions necessary for its function. Agentic systems need a related concept that is easier for ordinary users to understand: least capability. A Robot should have the smallest useful set of actions, resources and identities required for the task it performs.

The distinction is useful because an AI Agent does not merely access resources. It chooses actions. A coding Robot may need permission to read a repository and run tests. It may not need permission to push to the remote repository. A browser Robot may need permission to navigate an internal dashboard but not to approve a bank transfer. A meeting Robot may need to edit a shared diagram but not read unrelated project folders.

A useful design combines the Robot's configured authority, the task's working scope and the capability's permitted operations. The effective authority should be limited by all of them. These limits require runtime enforcement; a natural-language instruction alone does not create an access boundary.

Delegation should also have levels. A new Robot can begin in observation mode, where it analyzes but does not change state. It can move to proposal mode, where it creates patches or planned actions for approval. It can then receive bounded execution rights for well-understood operations. Finally, selected tasks may be allowed to run in the background without continuous supervision. The same Robot can occupy different levels for different actions.

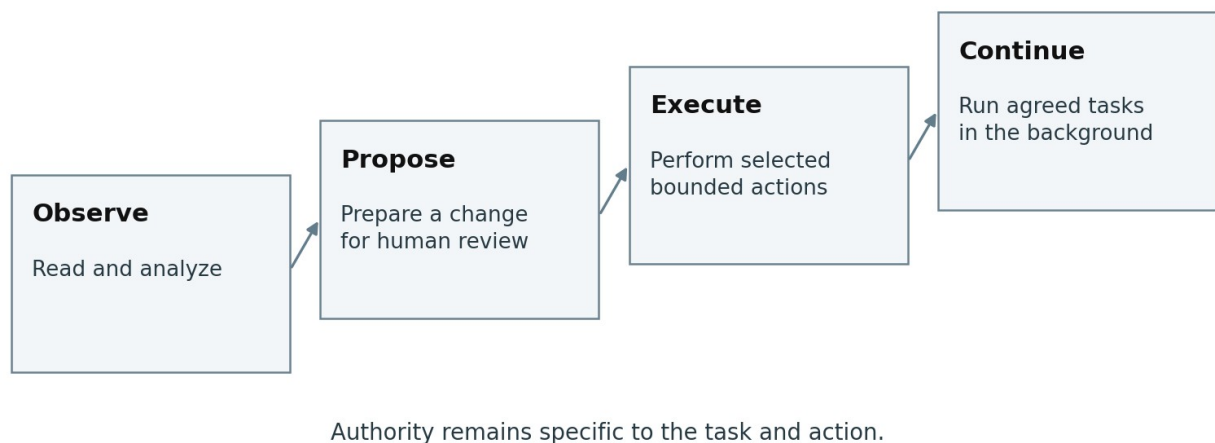


Figure 4. A ladder of delegation. Autonomy can increase gradually for selected actions rather than being treated as a single global setting.

Figure 4 shows this as a ladder rather than a switch. The platform does not force the user to choose between a powerless chatbot and a fully autonomous process. Trust can be increased gradually. More importantly, trust can remain asymmetric. A Robot may be highly autonomous in a test environment while permanently restricted in production.

The ladder has a human-factors advantage. Users are more willing to adopt automation when they can begin conservatively. A team can watch the Robot perform a workflow several times, inspect the results and then reduce approval requirements for the parts that have become predictable. This resembles how organizations delegate to new human colleagues, but the technical system can enforce the boundary more precisely.

Approval design must avoid fatigue. If the system asks for confirmation on every low-risk file read, users will learn to click through without thinking. Approvals should correspond to meaningful boundaries: sending externally, deleting irreversibly, spending money, publishing, changing production, using a protected identity, accessing a new sensitive folder. Low-risk actions should be governed by policy and performed without constant interruption.

A useful policy engine also explains why. “Denied by rule 7.4” is not an end-user message. “This Robot can prepare customer emails but cannot send them without your approval” is understandable. The technical mapping can remain available for administrators. As with the rest of AssistOS, the challenge is to hide complexity without hiding control.

Manage Robot identity and persistent sessions

A Robot that acts in real systems eventually needs identity. Identity is where the virtual-employee metaphor becomes operationally serious. If every Robot simply reuses the credentials of the human who launched it, then the system has no clean way to distinguish human action from Robot action, restrict authority by role or revoke access without disrupting the person.

Where possible, Robots should use dedicated service identities, scoped API tokens or delegated credentials. The organization can then know that the Web Operator accessed a procurement service while the Build Engineer did not. Access can be revoked per Robot. Audit records can identify the software actor. This is a more scalable model than copying personal API keys into configuration files.

Secrets should be treated as references, not ordinary memory. A Robot home can store configuration that says which secret it needs, while the secret value itself remains encrypted or managed by a dedicated mechanism. The runtime can inject the secret only into the environment where it is required. The Agent should not receive a list of unrelated secrets “just in case.” This reduces both accidental disclosure and prompt-injection risk.

Browser sessions are more difficult because authentication is often represented by cookies and local storage rather than explicit service tokens. A persistent browser profile may be necessary for practical automation, but it can contain authority equivalent to a password. AssistOS should therefore treat the browser profile as sensitive Robot state. It should be isolated from other Robots, encrypted at rest where appropriate and removable without deleting the rest of the Robot home.

The system should also support human-only authentication moments. Multifactor prompts, hardware security keys and legal consent screens may be inappropriate for autonomous completion. The Robot can pause, show the live browser, allow the user to take control, and then continue after the human step. This approach preserves usability without pretending that every identity mechanism should be automated.

Table 12. Identity patterns by task type

Term or aspect	Meaning or practical use
Dedicated service identity	Preferred when the target system supports role-oriented machine or service accounts.
Scoped token	Useful for APIs when rights can be limited to the exact operations a Robot needs.
Dedicated browser profile	Practical for browser-only systems, but the profile must be treated as sensitive Robot state.
Human takeover	Appropriate for MFA, hardware keys, legal consent or other steps that should remain explicitly human.
No external identity	Local-only analysis and deterministic tools may require no credential beyond the sandbox boundary.

The table emphasizes that there is no single best credential pattern. API services should prefer scoped service identities. Browser-only systems may require persistent profiles. Sensitive approvals may remain human. Local tools may need no external identity at all. AssistOS should provide the mechanisms while allowing organizations to choose according to risk.

Identity also intersects with meeting collaboration. A Robot joining WebMeet should have a visible identity distinct from the human who invited it. Other participants should know which software role

is present. If the Robot posts a message or edits a shared object, the action should be attributable to that identity. This avoids the subtle failure mode in which AI-generated work appears under a human participant’s name and becomes impossible to distinguish later.

The long-term research around decentralized identity and OpenDSU may provide additional ways to carry identity and provenance across organizational boundaries. That belongs to the future-facing part of AssistOS, not to the minimum viable product. The immediate product requirement is simpler: software workers should not be anonymous extensions of human credentials.

Inspect actions and recover from failure

Autonomous systems become dangerous when the only state visible to the user is “thinking.” A Robot platform needs observability at the level of work. The user should know what task is running, what stage it has reached, whether it is waiting, what resources it is consuming and what artifacts it has changed. Administrators may need deeper logs, but the ordinary user needs an operational picture.

Observability is distinct from exposing internal chain-of-thought. Modern AI systems may have private reasoning processes that should not be revealed, and raw reasoning traces are often too verbose to be useful. AssistOS can instead record externally meaningful events: plan accepted, file opened, tool invoked, test completed, approval requested, browser takeover started, output written. The objective is accountability for action, not surveillance of hidden cognition.

Task status also supports intervention. A user who sees “waiting for approval to send” can respond immediately. A user who sees “retrying test 12 of 14 after dependency failure” may decide to let the Robot continue. A user who sees no progress for twenty minutes can stop the task. These states turn agent supervision into an ordinary interface problem.

For enterprises, audit records can associate significant actions with Robot identity, human initiator, capability version and policy context. The appropriate retention period and detail depend on the organization. High-volume low-risk events may be summarized. Security-sensitive actions may require durable records. AssistOS should provide the structure without assuming that one retention policy fits every sector.

Recovery is the other half of observability. A platform that records a mistake but cannot recover from it is only partially useful. Task workspaces should support rollback where the artifact type allows it. Code can use version control. Documents can preserve versions. Browser actions may be irreversible, which is why risky submissions should have stricter approval. Containers can be rebuilt. Robot homes can be backed up separately from disposable runtime state.

Table 13. Recovery options depend on whether the action is reversible

Term or aspect	Meaning or practical use
Versioned file edit	Restore the previous version or reject the Robot-produced revision.
Code task	Discard the worktree or branch; preserve the diff and test evidence for review.
Sandbox failure	Rebuild the runtime while preserving the meaningful Robot home and project artifacts.
Browser navigation	Stop or reset the session; changes already submitted to an external service may not be reversible.
External transaction	Often irreversible or costly to reverse; use stronger human approval before execution.

The recovery table makes an important distinction between reversible and irreversible actions. A file edit is often easy to revert. A published message or financial transaction may not be. The platform should use this difference when deciding which actions can be automated freely. Autonomy is safest where reversibility is high.

Observability also improves product development. Aggregate failure patterns reveal which capabilities are immature. If many Robots fail after a Ploinky Agent update, the package can be rolled back. If a browser workflow breaks after a website redesign, the system can mark the task as degraded rather than silently continuing. A platform can become more reliable because it sees its own operational behavior.

The result is a different trust model from conversational confidence. Users do not need to trust that the Robot “understands” everything. They need to know that it operates within boundaries, produces inspectable artifacts, reports failures and can be stopped or recovered when necessary. This form of trust is less anthropomorphic and more compatible with enterprise engineering.

Govern Robots across an organization

A company evaluating AssistOS will ask questions that individual enthusiasts can postpone. Who can create Robots? Which Ploinky Agents are approved? Which models may process confidential data? Can one department install a capability that opens network access? How are secrets managed? Where are task histories stored? Can an administrator disable a compromised Robot immediately? These questions are not bureaucracy around the product. They are the product at enterprise scale.

The platform should support an organizational catalog of approved capabilities. Developers can experiment in a development environment, while production users receive curated versions. Policies can restrict external model providers, network destinations and data classes. Role-based or attribute-based access can control who may attach a Robot to a project. Cost policies can prevent a background task from consuming unlimited remote inference.

Governance should also be compatible with self-hosting. Some organizations will accept a managed cloud service. Others will require private deployment because of data residency, internal network access or sovereignty policy. AssistOS gains strategic flexibility by separating the open runtime and project model from optional commercial management layers.

The failure modes are broader than classic cybersecurity. Prompt injection can manipulate an Agent through untrusted text. Hallucination can cause the Robot to act on a false assumption. Tool misuse can transform a small reasoning error into a real-world action. Dependency supply chains can introduce malicious software. Browser automation can encounter deceptive interfaces. Long-running tasks can accumulate cost. Model provider outages can interrupt work. A marketplace can distribute low-quality or unsafe packages. A governance design must address this range rather than treating “AI safety” as one problem.

Table 14. Important failure modes and corresponding design responses

Term or aspect	Meaning or practical use
Prompt injection	Limit tool authority, isolate untrusted content and avoid allowing text alone to expand permissions.
Hallucinated assumption	Require evidence or deterministic validation when facts have operational consequences.
Tool misuse	Use narrow tool contracts, task scopes and approval boundaries.

Term or aspect	Meaning or practical use
Dependency compromise	Sign packages, scan dependencies, track provenance and support revocation or rollback.
Cost runaway	Apply task and Robot budgets for model calls, compute, storage and external services.
Interface drift	Detect changed websites, CLIs or APIs and fail visibly rather than improvising destructive actions.
Human over-trust	Show role, status, evidence and limits; avoid persuasive anthropomorphism as a substitute for performance history.

The table places each failure beside a design response, but it is deliberately not a guarantee. No platform can prove in advance that every future agent, model or dependency is safe. The realistic objective is risk reduction through layered controls, testing and clear recovery paths.

There is also a category risk: users may over-trust software because it is called an employee or given a name. AssistOS should counter this through interface design. Robots can have roles without pretending to be people. The system can show model uncertainty when relevant, require evidence for claims and keep approvals visible. A Robot should earn confidence through performance history, not through a persuasive persona.

For an investor, the governance layer is one of the strongest arguments that AssistOS can become more durable than a thin AI application. As models improve, the need to decide which software worker may act on which data does not disappear. It becomes more important. The platform can accumulate value in policy, packaging, identity, history and integration even if the underlying intelligence becomes cheaper.

For a buyer, the same point can be stated more simply. AssistOS should make AI delegation behave like managed computing rather than like handing a password to a clever stranger. The system will never eliminate risk, but it can make authority explicit, action visible and failure recoverable. That is the threshold at which virtual workers become an infrastructure question rather than a novelty.

Chapter 5 Understanding the machinery that supports the work

Robots need tools, dependencies, interfaces and persistent state as well as models. This chapter explains those implementation choices after establishing what the person needs to do.

Provide the tools that a model needs

It is tempting to think that an AI platform is mostly a model-selection problem. Choose a strong model, provide a prompt, connect a few APIs, and the application is nearly finished. This is often sufficient for a demonstration. It is rarely sufficient for a working environment in which software is expected to operate for hours, manipulate files, install dependencies, interact with external services and remain safe enough to be reused by other people.

A practical Robot needs capabilities that models do not provide by themselves. A coding Robot needs a shell, compiler, package manager, repository tools and tests. A data Robot may need Python libraries, database drivers, command-line utilities and visualization software. A browser Robot needs a browser runtime, perhaps a virtual desktop, and a place to keep its profile. A research Robot may need PDF utilities, citation databases and specialized retrieval tools. These dependencies are not merely conveniences. They are part of the operational definition of the task.

They also create risk. Installing arbitrary packages directly on a host machine is difficult to govern. Different tools require conflicting versions. One agent may need Node.js while another needs Python and system libraries. Some tools need background services. Some require access to graphics or network devices. An experimental coding agent may execute shell commands that would be unacceptable on the host. A platform that ignores these concerns transfers the burden to every user.

Ploinky addresses deployment through packages that declare their runtime, dependencies, startup commands and interfaces. Its Agent Specification documents container execution and optional host sandbox mechanisms, including bubblewrap on Linux and seatbelt on macOS. This supplies a foundation for repeatable capability installation; it does not by itself implement every Robot or enterprise feature described in this handbook. [P8]

A Ploinky Agent can be small. It may wrap a command-line program and expose it through WebChat or MCP. It can also be substantial. It may include a coding agent plus the language runtimes it needs. It may start a web application. It may host a service that exposes an OpenAI-compatible endpoint. It may install a complete scientific toolchain whose dependencies would be painful to reproduce manually. The packaging boundary allows AssistOS to treat all of these as capabilities that can be enabled, updated and isolated.

Table 15. Why packaging matters even when the AI model is excellent

Term or aspect	Meaning or practical use
Repeatability	The same capability can be installed again from a declared runtime rather than reconstructed manually.
Isolation	Dependencies and commands run inside a controlled environment instead of directly on the host.
Dependencies	System packages, language runtimes and services can be bundled with the capability that needs them.

Term or aspect	Meaning or practical use
Lifecycle	Capabilities can be started, stopped, upgraded, rolled back and associated with explicit versions.
Distribution	A useful research repository can become an installable product rather than a set of setup instructions.
Evaluation	The package provides a stable unit against which functionality, security and resource use can be tested.

A package can move from a developer's environment to a user's installation through a declared contract. Versioning, permissions, resource needs, health checks, evaluations and support information can make that contract useful throughout the capability's lifetime. The distribution model in Chapter 6 depends on this operational detail.

Packaging also protects model neutrality. A Robot may use one remote model today and a local model tomorrow, but the rest of its environment can remain stable. Conversely, a team may keep the same model while replacing the coding agent that orchestrates it. The platform becomes a place where these substitutions are operationally ordinary. This is a more durable value proposition than betting that one model or one agent framework will dominate permanently.

The user should mostly not see Ploinky. A person creating a Research Robot should not have to choose container networks or inspect a manifest unless they want to. The platform should present the consequences: this capability needs 8 GB of storage, can access these folders, requires network access to these domains and will install these components. A developer or administrator can open the technical details. Good infrastructure is visible when control matters and invisible when it does not.

Package a deployable capability

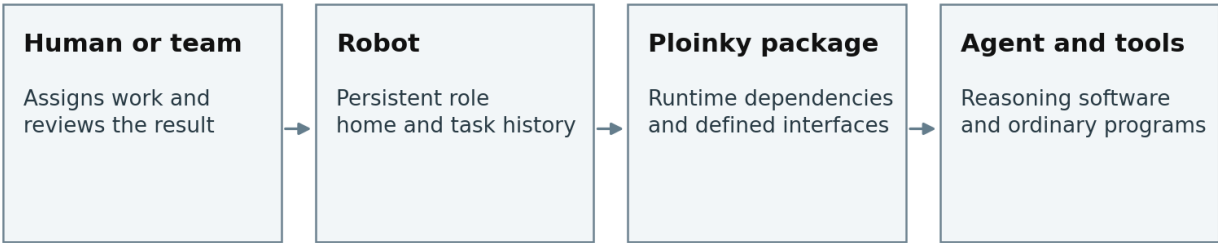
The term “Ploinky Agent” can be confusing because the package does not need to contain an autonomous AI agent. Ploinky’s own documentation intentionally uses “agent” broadly for a tool that can be installed and run. For AssistOS users, it is more useful to think of a Ploinky Agent as a deployable capability unit. It is the smallest thing that the runtime knows how to install, start and expose as a coherent service.

Consider three examples. A `codexAgent` package may install a coding agent, its command-line environment and the basic tools needed to inspect a repository. A `browserAgent` may provide a browser and a computer-use interface. A `pdfAnalysis` package may contain no autonomous reasoner at all; it may simply expose robust PDF extraction, rendering and conversion tools through MCP. From the Robot’s perspective, all three are capabilities. One reasons, one operates an interface, one performs deterministic transformations.

This approach is important because trustworthy systems should not use an LLM for work that ordinary software can do better. If a PDF library can extract metadata deterministically, the Robot should call that tool rather than ask a language model to infer the metadata from a screenshot. If a cryptographic library can verify a signature, the model should not “reason” about whether the signature looks valid. Agents are valuable orchestrators precisely because they can combine deterministic tools with probabilistic reasoning.

Ploinky Agents can expose different interaction contracts. MCP is especially suitable for tools and data because it standardizes how AI applications discover and call external capabilities. The official MCP documentation describes it as an open standard for connecting AI applications to data sources,

tools and workflows. An OpenAI-compatible API can be useful when a service naturally behaves like a language or multimodal model endpoint. A CLI remains valuable for mature developer tools. A web service may be appropriate for an application with its own interface. Ploinky’s job is to package the runtime, not to force every capability into one protocol.



The role and project can persist when an engine or package changes.

Figure 5. The core mental model. Humans delegate to Robots; Robots use deployable Ploinky Agents; those packages may contain AI Agents and deterministic tools.

Figure 5 places the deployable capability in context. A human delegates to a Robot. The Robot uses one or more Ploinky Agents. A Ploinky Agent may contain an AI Agent, deterministic tools, or both. The AI Agent may in turn call MCP tools or other APIs. The direction of delegation is distinct from the direction of packaging, which is why collapsing everything into the word “agent” causes confusion.

The package also provides a useful point for lifecycle management. A capability can have a version. It can declare prerequisites. It can be enabled for one workspace and disabled for another. A new version can be tested before broad deployment. The system can associate task history with the version that actually ran. These details become essential once a company depends on Robots for real work.

A Ploinky Agent should also be evaluable. The fact that a package starts successfully does not mean that it performs its advertised task. A research package could include a basic evaluation suite. An enterprise catalog could record which versions have passed internal tests. A public marketplace could eventually show supported environments, expected resource consumption and benchmark results. This is where distribution can become a trust mechanism rather than a popularity contest.

The long-term value of this layer is subtle. AI capabilities are becoming abundant, but abundance creates integration cost. There may be thousands of repositories that can perform some useful operation, yet only a small fraction are packaged in a way that an ordinary organization can install, govern and support. Ploinky’s opportunity is to turn raw capability into an operational unit.

Give each Robot a persistent home

The Robot home is one of the simplest ideas in AssistOS and one of the most important. A human user account has a home directory because programs need a persistent place for personal configuration and state. A virtual worker has the same need. If the Research Robot has to relearn every preference, redownload every utility and reauthenticate every website for every task, it will be

expensive and brittle. If all Robots share one global state directory, they will interfere with one another and inherit more authority than they need.

The private sandbox gives that home a controlled place to be used. Installing an agent once should allow several Robot configurations to start from the same capability while retaining separate writable environments. Creating a second Robot should not silently copy the first Robot's credentials or grant access to its projects. Stopping or rebuilding an execution instance should preserve the Robot's selected durable state; sharing a project should be an explicit permission, not a consequence of sharing an agent installation.

A Robot home creates a durable namespace. It can contain configuration files for the primary Agent, tool preferences, caches, local indexes and user-approved persistent application state. A browser Robot can keep a dedicated browser profile there. A coding Robot can keep agent-specific configuration without writing it into the host user's personal home. A domain Robot can preserve dictionaries or templates that belong to its role.

The persistence boundary should be designed carefully. There are at least four categories of state that are easy to conflate. Role configuration describes how the Robot should behave. Cached data can be reconstructed and therefore can often be deleted safely. Project artifacts belong in projects, not hidden inside the Robot home. Secrets such as API keys and authentication tokens require stronger protection and should ideally be referenced through a secrets mechanism rather than stored as ordinary files. Browser cookies sit awkwardly between configuration and secrets because they can embody authenticated authority.

Table 16. What belongs in a Robot home, and what should remain separate

Term or aspect	Meaning or practical use
Role configuration	Prompts, preferences, agent configuration and other persistent settings that define the worker role.
Rebuildable cache	Indexes, downloaded assets and intermediate state that can be regenerated when necessary.
Project artifacts	Usually stay in project folders so they remain visible, versioned and portable independently of the Robot.
Secrets	Prefer references to protected secrets or scoped injection; do not treat API keys as ordinary memory.
Browser profile	Persistent cookies and local storage may be necessary, but should be isolated and governed as sensitive authority.

The table makes the design recommendation explicit: persistence is useful, but persistence should have classes. A Robot home should not become a convenient dumping ground for every object the Robot touches. A user should be able to reset cached state without deleting project work. An administrator should be able to revoke credentials without erasing the Robot's configuration. A Robot should be replaceable while its project outputs remain intact.

The home also helps make multiple Robots genuinely different. Two coding Robots can use the same underlying coding Agent but maintain different roles, prompts, tool preferences and project access. One may be a conservative reviewer that never writes to production branches. Another may be a prototyping Robot that works aggressively in disposable sandboxes. Their underlying AI capability can be identical while their durable behavior is not.

There is a parallel with service accounts in enterprise systems. Organizations rarely want every automated process to execute with a human employee's unrestricted identity. They create scoped

identities for services. A Robot is richer than a service account because it has an interactive role and workspace, but the security lesson is similar. The Robot should have an identity and home that are distinct from the human who created it.

Persistence also improves debugging. If a task behaves differently today than yesterday, the user can inspect the Robot’s relevant configuration and capability version. If the entire environment is ephemeral and opaque, reproducibility is difficult. At the other extreme, if state is never reset, subtle corruption accumulates. AssistOS should support controlled rebuilding: preserve the meaningful Robot home, recreate the sandbox and reattach allowed state. A durable worker should not require an immortal container.

This last distinction is important architecturally. The Robot is not identical to one container instance. Containers can be stopped, upgraded or replaced. The Robot is the durable identity and state model that can be backed by one or more sandbox executions over time. That separation makes the system more resilient and avoids turning implementation detail into user identity.

Connect tools through suitable interfaces

A platform that packages capabilities still needs a common way to use them. No single interface is appropriate for every tool, which is why AssistOS benefits from supporting several interaction contracts. The objective is interoperability without flattening useful differences.

MCP is a natural fit for tool invocation. It allows an AI application to discover named tools and call them with structured arguments. A Ploinky Agent can expose an MCP surface for file operations, data queries, search, compilation, conversion or specialized domain logic. The Robot’s AI Agent does not need to know how the tool is implemented internally. It only needs the contract and the permission to call it.

OpenAI-compatible APIs serve a different purpose. Many model servers and agent systems implement interfaces inspired by OpenAI’s chat or response conventions because the ecosystem already knows how to integrate them. A Ploinky Agent can wrap a local model server or specialized reasoning service behind such an interface. Compatibility here is pragmatic rather than ideological. It reduces integration friction and allows capabilities to be substituted more easily.

Command-line interfaces remain extremely valuable, especially for coding and engineering. Tools such as Codex and Claude Code are designed around a terminal environment and can directly operate repositories and shell commands. Ploinky can install such tools inside isolated environments and expose their CLI through WebChat or a Robot task. The fact that the human sees a chat interface does not mean the underlying agent has to be a web application.

Long-running web services and full web applications are also legitimate Ploinky Agents. A data visualization tool may already have a mature web UI. A scientific system may expose a local dashboard. AssistOS does not gain anything by rewriting those interfaces as chat. Explorer can embed or route to the web surface while Ploinky manages the isolation and lifecycle beneath it.

Table 17. Different interfaces serve different kinds of capability

Term or aspect	Meaning or practical use
MCP	Best suited to structured tools, resources and workflows discoverable by AI applications.
OpenAI-compatible API	Useful for model-like or conversational services that benefit from a widely supported de facto request format.

Term or aspect	Meaning or practical use
CLI	Natural for coding agents and mature developer tools that already operate through a terminal.
Web service / UI	Appropriate when a capability has a long-running service or a valuable visual interface of its own.

The practical design rule is to preserve the native strength of the capability. Use MCP when structured tool calling is the natural interface. Use an OpenAI-compatible endpoint when the capability behaves like a model or conversational service. Use a CLI for mature command-line agents. Use a web surface when visual interaction is central. A Robot can coordinate across these interfaces through the same working context.

This pluralism also reduces vendor lock-in. A specialized service can be replaced behind an interface contract. A Robot can switch from one coding agent to another without changing the entire project model. A local model server can replace a remote endpoint for selected work. The interfaces become seams where the system can evolve.

There are limits. “Compatible” APIs often differ at the edges. MCP tools can have inconsistent semantics. CLIs can change flags. Web applications can break automation when their interface changes. The runtime therefore needs adapter testing and version awareness. Interoperability is an engineering discipline, not a declaration. AssistOS should prefer a small number of well-supported contracts and test them continuously.

The benefit to the user is that these differences become implementation choices. The user asks the Robot to perform a task. The platform chooses or follows a configured capability path. When a choice matters for cost, privacy or outcome quality, the user can see it. Otherwise the platform should avoid exposing protocol trivia as part of ordinary work.

Choose where the work runs

AssistOS presents itself as local-first, but local-first should not be confused with local-only. The useful principle is that the location of computation remains a policy choice. Some work can run comfortably on a laptop. Some requires a workstation with a GPU. Some belongs on a private organizational server. Some legitimately uses remote model APIs or cloud infrastructure. A mature platform coordinates these locations without making the project depend on one of them.

Local execution offers several benefits. It can reduce latency for small operations, keep sensitive files close to the user, continue working through network disruptions and make marginal compute costs predictable once hardware is owned. It also makes persistent browser and desktop automation easier to reason about because the execution environment can remain physically near the user or organization.

Local inference is already a practical route for selected workloads. Apple's Foundation Models framework exposes an on-device language model for tasks such as summarization and extraction, with structured generation and tool calling; Microsoft's Phi work demonstrates small-model deployment on PCs. [E13, E14] These examples motivate the placement choice, not a dependency on those vendors. AssistOS should be able to use a local model through a supported interface, evaluate it on the project's task and reserve remote inference for work that actually needs it.

Cloud execution offers different benefits. Large models may be impractical to run locally. Bursty workloads benefit from elastic compute. Distributed teams may want centrally managed Robot environments. Specialized services may exist only as hosted APIs. The sensible architecture is

therefore hybrid: run locally when it is sufficient and valuable, escalate to remote resources when there is a reason, and make that reason visible to policy.

Ploinky's packaging model helps because the same capability can, in principle, be deployed to different execution targets when its dependencies allow. A container that runs on a developer workstation can also run on a managed server. A lightweight sandbox may be preferable for a small trusted utility, while a stronger container boundary is appropriate for a coding agent installing arbitrary packages. The platform can choose isolation based on risk and environment rather than forcing every capability into the same cost profile.

Placement should follow the whole task's dependencies. A local document extractor may still call a remote model later. A background Robot running on a laptop cannot continue after that machine is shut down unless execution has been explicitly transferred or restarted elsewhere. Likewise, a model fallback is useful only if it has the required tools and has passed the workflow's checks. Keeping these dependencies visible makes local, server and cloud execution practical choices rather than broad labels.

Deployment location is also related to data movement. A strong local-first design avoids the false choice between sending an entire project to the cloud and refusing all remote intelligence. A Robot may perform local preprocessing, redact or select relevant fragments, and send only the information required for a remote reasoning step. This pattern is not automatically private; it must be designed and validated. But it provides a more nuanced architecture than cloud-only applications that require wholesale upload.

For enterprises, central administration can coexist with local execution. Policy, catalogs and identity can be managed centrally while task execution occurs on approved local machines or private servers. The organization can choose which Ploinky Agents are available, which model routes are permitted and what network egress each Robot receives. Employees experience a coherent workspace rather than a collection of independent security products.

The deployment story should remain incremental. A single user should be able to start with one machine. A small team may add a shared server. A larger company may deploy managed Ploinky infrastructure and organizational policy. The product is stronger when the first useful experience does not require enterprise architecture, but the same conceptual model can scale into enterprise controls.

This is the larger meaning of the “operating layer” thesis. AssistOS does not need to replace the host operating system. It coordinates a new class of software workers across existing systems. When the Robot identity, permissions, project memory and capability contracts survive a move from laptop to server or from one model to another, the platform begins to function like an operating layer for intelligence.

Chapter 6 Developing the wider platform

The operating-layer thesis follows from daily use: work needs continuity across replaceable models, tools and deployments. This chapter connects that requirement to distribution, commercial development and a limited research horizon.

Understand AssistOS as an operating layer

Natural language gives people a convenient way to ask software to do work. Sustained work also needs artifacts, versions, identities, permissions, running tasks and review decisions. AssistOS brings these elements into a project that can persist beyond any one conversation.

AssistOS can be understood as the attempt to add those structures without abandoning natural language. WebChat remains the easiest way to express intent. Explorer gives the intent a place to operate. Robots give delegated work a persistent identity. Ploinky provides installable capabilities. WebMeet lets people and Robots collaborate around shared state. Security and observability constrain action. The “operating layer” is the combination.

This is analogous to earlier computing transitions in a limited but useful sense. Raw compute became broadly useful when operating systems made hardware manageable. Networked documents became broadly useful when browsers made protocols navigable. Cloud infrastructure became broadly programmable when virtualization and orchestration standardized deployment. AI models are a raw capability of extraordinary breadth, but organizations still need a layer that turns the capability into managed work.

Linux provides a useful precedent: an open foundation can support independent installations, competing service providers and very different uses over decades. We expect a substantial locally operable AI ecosystem to emerge for similar reasons of control, adaptability and continuity. AssistOS is intended as an operating layer within that ecosystem. This is a strategic expectation, not a claim that AssistOS is destined to become its dominant platform. Its contribution must be earned through useful workflows that citizens and organizations can operate independently.

The durable object in this thesis is the project. Models change quickly. Agent frameworks change quickly. A project may last years. The specifications, documents, decisions and histories accumulated there should not disappear because a supplier changes. AssistOS can create continuity around work while treating intelligence as replaceable.

Table 18. From an AI feature to an operating layer

Term or aspect	Meaning or practical use
Durable object	Thin feature: conversation. Operating layer: project and its artifacts.
Worker identity	Thin feature: transient assistant session. Operating layer: persistent Robot with home and policy.
Model portability	Thin feature: often vendor-specific. Operating layer: models and agents are replaceable capabilities.
Permissions	Thin feature: application-level access. Operating layer: Robot-, task- and capability-scoped authority.
Asynchronous work	Thin feature: mainly synchronous interaction. Operating layer: managed background tasks and visible status.

Term or aspect	Meaning or practical use
Review	Thin feature: prose response. Operating layer: artifact-native diffs, versions, tests and evidence.

The table contrasts a thin AI feature with an operating layer. The difference is not the amount of code. It is where continuity and control live. A prompt wrapper depends heavily on one interaction. An operating layer owns the relationship among artifacts, permissions, tasks and capabilities while allowing the model beneath to evolve.

This position explains why AssistOS should integrate strong tools into a coherent workplace. The ambition is an IDE for computer-based activity: shared project context, editing, execution, inspection, testing and review, available through AI copilots and personalized Robots. An office editor, scientific notebook or video conference can remain a specialized application while participating in the same task and artifact history. The operating layer supplies continuity across them.

If the strategy works, users may eventually open AssistOS not because they want to “use AI,” but because that is where their team’s work and virtual workers live. At that point the operating-layer description becomes a consequence of habit. The product has earned the category through daily utility.

Preserve control and continuity

“Sovereignty” can sound political or abstract, but in a computing product it can be stated through concrete rights. A user should be able to know where important data is stored. An organization should be able to decide which models receive which data. Projects should remain exportable. Capabilities should be replaceable. A local or private deployment should be possible when control matters. The platform should not require users to surrender these choices merely to gain AI assistance.

Local-first design supports these rights but does not require ideological rejection of the cloud. A small model running locally may be ideal for classification, extraction or private retrieval. A frontier remote model may be justified for a difficult reasoning task. A company may run Ploinky on private servers while using a hosted video service. The platform can treat location as a policy variable.

Model neutrality is equally important. The rate of model improvement makes long-term dependency on one model technically risky. A Robot role should outlive the engine currently configured for it. A company should be able to evaluate a new model against existing tasks. The project history can provide realistic evaluation cases. If the new model performs better or reduces cost, it can be adopted without rebuilding the user’s working environment.

A practical continuity test is to change the model for a representative task and inspect what survives. The source files, accepted versions, criteria and previous results should remain accessible. The new run should record its model choice and pass the same checks. A second test is to restore the project in another environment and identify the missing capabilities. Exporting documents while losing the procedure, permissions or dependency information preserves only part of the work. [P6, Chapter 8]

Privacy and efficiency can benefit from the same division of work. A local model can interpret a request, a local index can retrieve permitted evidence, and a symbolic or conventional program can apply explicit rules without transmitting the entire project to a large remote model. The resulting system should be evaluated for accuracy, human correction time, compute use and actual data

movement. A smaller model is valuable when the complete workflow meets its requirements; merely replacing a large model does not establish either trustworthiness or privacy.

This neutrality is not free. Models expose different tool-calling behavior, context limits, multimodal features and safety constraints. OpenAI-compatible interfaces reduce some friction but do not erase semantic differences. AssistOS therefore needs evaluation and adapters, not merely endpoint switching. The product can make model replacement possible without pretending it is trivial.

Sovereignty also includes the ability to inspect a process. An organization can own its files yet remain dependent if only its supplier can reconstruct an important workflow. Procedures, execution histories and provenance preserve knowledge of how the work was done. [P6, Chapter 8]

Table 19. Sovereignty translated into product properties

Term or aspect	Meaning or practical use
Data location	The user or organization can determine where important project data and execution state live.
Model choice	Remote and local models can be selected according to cost, capability and policy.
Exportability	Projects and artifacts can leave the platform in usable forms.
Private deployment	Core workloads can run on user-controlled machines or servers when required.
Process traceability	Important outputs can be linked to tasks, capability versions, approvals and source artifacts.

The table translates sovereignty into product properties. None requires every workload to stay on one laptop. They require that location, provider and export remain choices governed by the user or organization. This is a more practical definition than treating sovereignty as isolation.

An exit path should declare its limits. Licensed data may not be transferable, a hosted integration may still require an account and a private model may be unavailable on a smaller device. The user needs to know which functions can continue, which degrade and which require an alternative. That is a more useful standard of portability than assuming that every dependency can be moved unchanged.

For enterprises, this positioning can be strategically important in Europe and other jurisdictions where data governance, vendor concentration and digital autonomy are active concerns. AssistOS should avoid making legal compliance claims without validation, but a self-hostable and model-neutral architecture can provide useful ingredients for organizations that need control over processing location and data flows.

For individuals, sovereignty is more personal. A family archive, private writing project or personal research library should not become unusable because one subscription ends. A Robot that organizes those materials can run locally when appropriate. The project can remain on the user’s storage. The value is continuity more than ideology.

The strongest version of the idea is simple: the intelligence provider can change while the work remains. That principle is likely to survive changes in models, interfaces and infrastructure, which is why it belongs near the center of AssistOS’s long-term identity.

Develop a useful capability catalog

The AI ecosystem has a distribution paradox. Useful code is abundant, yet adopting it is difficult. A research repository may implement a strong method while requiring exact dependency versions, obscure hardware setup and manual configuration. An MCP server may expose valuable tools but provide little information about security or maintenance. A coding Agent may evolve weekly. Users face a scarcity of confidence rather than a scarcity of software.

Ploinky packaging can create the basis for a capability catalog. A package can declare what it installs, how it starts, which interfaces it exposes and which resources it needs. AssistOS can add evaluation, permissions and lifecycle information. An organization can curate a private catalog. A public marketplace can eventually distribute trusted packages and plugins.

The order matters. A marketplace should not be the first product. Thousands of poorly described agents would reproduce the chaos the platform is trying to solve. The initial catalog should contain a small number of excellent first-party or carefully curated capabilities that solve real workflows. Users join because the product works. Developers join because users exist. The marketplace becomes a scaling mechanism after adoption rather than a substitute for adoption.

A mature package contract could include outcome-oriented metadata. What problem does the capability solve? What kind of input does it expect? Which models are compatible? What resource footprint is typical? Which network destinations are required? Which permissions are sensitive? What evaluations were run? What license and support terms apply? This turns installation into an informed decision.

Table 20. A capability marketplace needs an operational contract

Term or aspect	Meaning or practical use
Outcome	A plain description of the problem the capability solves.
Runtime	Container or sandbox requirements, supported platforms and dependencies.
Interfaces	MCP, API, CLI, web service or other exposed contracts.
Permissions	Files, devices, identities and elevated actions the capability can request.
Network	Expected external and internal destinations.
Resources	Typical storage, memory, CPU/GPU and model requirements.
Evaluation	Functional, safety or domain tests associated with the package version.
Version and support	Release history, maintainer information, update policy and removal path.

Operational confidence comes from packaging, testing, compatibility, update paths and accountable maintenance. The catalog can help users compare those properties before enabling a capability. A listing without this information provides little assurance about whether the software will perform well in a particular workspace.

Commercial models can emerge around managed deployment, enterprise support, private catalogs, certification and vertical distributions. A research organization may buy a curated Research Workstation. An engineering team may buy managed coding Robots and governance. A regulated company may buy validation and support around a narrower set of capabilities. Open source and commercial value are not contradictory when the commercial value lies in operation and trust.

Large platform vendors can bundle agent infrastructure into products that organizations already use. AssistOS therefore needs a clear reason to be adopted: useful coordination across providers, local

and private deployment where required, and continuity around project artifacts. These properties have to produce a better working experience, not merely a longer feature list.

Another risk is quality control. A marketplace can become a supply-chain attack surface. Packages need signatures, provenance and scanning. Evaluations need to be reproducible. Revocation and update channels need to exist. A capability that runs arbitrary code should not receive the same trust treatment as a read-only formatter. The marketplace is as much a security institution as a distribution mechanism.

The opportunity is significant if executed carefully. As raw AI capability becomes cheaper, users may pay less for access to intelligence itself and more for reliable ways to apply it. Packaging is where the messy diversity of the ecosystem can become a product.

Build commercial value around deployment and maintenance

A useful open-source capability and a dependable product are separated by work. Someone must package the dependencies, test the supported environments, explain the permissions, maintain the integration and help users when a deployment fails. This gives maintainers and service providers something concrete to offer beyond access to a model. [P6, Chapters 2 and 10]

Consider a team that has developed a good document comparison method. Publishing the repository lets other developers inspect it, but a nontechnical organization still needs a usable installation and a supported workflow. A Ploinky package can carry the runtime; an Explorer plugin can provide the comparison view; a Robot configuration can supply the role and task defaults. Evaluations explain where the method works, and an accountable maintainer handles updates. These components turn a method into something that a buyer can operate.

The first distribution can remain small. A research workspace might combine source management, analysis tools, a writing capability and a reviewer. An engineering workspace might focus on bounded code changes and test evidence. These are proposed starting configurations, not separate completed product lines. They should share the same project and Robot model so that domain specialization does not fragment the platform.

An organization may pay for installation, managed operation, private catalogs, integrations or a defined support relationship. The benefit must be observable: less internal effort maintaining environments, faster recovery from an update, reliable access controls or lower cost per accepted task. Open-source availability supports inspection and the possibility of changing provider. It does not imply that every model, dataset, plugin or external service has the same license or commercial terms.

For maintainers, a catalog becomes useful when it brings real users and feedback, rather than merely another place to list a repository. For users, it becomes useful when entries explain outcomes, limitations and operational requirements. Neither side benefits much from a large count of packages that cannot be installed or evaluated consistently.

The commercial sequence should follow the work: establish a few useful workflows, make deployments repeatable, then expand support and distribution. Compatibility knowledge and evaluation experience can accumulate across those deployments. They become valuable only when they reduce a user's effort or uncertainty. The enterprise Linux analogy helps explain this service around an open foundation, but it does not establish demand for AssistOS; demand still has to be demonstrated through sustained use.

Assess the strategic proposition

A possible investor evaluating AssistOS should begin with the user story, not the research portfolio. The question is whether individuals and teams will adopt persistent virtual workers that operate inside a managed workspace. If that behavior does not become useful, the larger operating-layer thesis has no foundation. If it does become useful, several durable value layers become possible around deployment, identity, governance, project memory and capability distribution.

The strongest strategic argument is that intelligence is becoming more interchangeable while operational context is not. Model capability may commoditize unevenly, but companies will still need to know which worker can access which project, which tool version ran, what was approved and where the result lives. AssistOS can own this continuity without owning the underlying models.

The initial product should therefore prioritize a small number of exceptional workflows. A coding Robot that reliably completes bounded background tasks is more valuable than a marketplace of experimental agents. A Browser Robot with smooth human takeover is more valuable than a grand claim about autonomous enterprise. A WebMeet demonstration in which a Robot visibly changes a shared artifact can make the collaboration thesis concrete. Product truth should lead research narrative.

Table 21. The AssistOS proposition in investor language

Term or aspect	Meaning or practical use
User problem	AI can answer, but teams still lack a stable place to delegate, supervise and preserve AI-performed work.
Initial wedge	Coding, research, document work and observable browser automation where outputs are reviewable.
Why now	Coding agents, computer-use systems, MCP and local inference make persistent software workers technically plausible.
Durable value	Robot identity, project continuity, governance, packaging, integration and operational history.
Differentiation	Open, model-neutral, local/private-capable coordination across heterogeneous agents and tools.
Principal risks	Security, complexity, platform bundling, weak product focus and unreliable agent behavior.
Milestones	Reliable bounded tasks, smooth intervention, team collaboration, enterprise governance, then ecosystem expansion.
Business model	Managed deployment, enterprise controls, support, certified capabilities, private catalogs and vertical distributions.

The strategic assessment should connect adoption, repeated deployment and cost per accepted task. A convincing demonstration establishes a workflow worth testing further. Repeated use across representative cases establishes stronger evidence. Neither alone establishes a scalable business.

The principal execution risk is complexity. The platform contains many useful technical ideas, but users need a stable vocabulary and a small number of understandable actions. Product development should preserve the relationship among project, Robot, task and result while improving the machinery underneath.

Security is another existential risk. A platform that gives software workers persistent browser sessions and shell access will attract scrutiny. A serious enterprise product must invest in isolation,

secret handling, policy, update security, logging and independent testing. Security cannot be a later feature because trust is the market category.

The business model is credible but should follow usage. Commercial value can come from enterprise management, support, private deployment, certified packages, vertical distributions and managed infrastructure. A public open-source core can accelerate ecosystem adoption if the boundaries are designed carefully. The company should avoid depending entirely on per-token resale margins that model providers can compress.

The milestone sequence can be concrete. First, make one user with a few Robots productive on one machine. Second, make background, browser and computer-use tasks observable and interruptible. Third, make team collaboration through Explorer and WebMeet coherent. Fourth, add enterprise identity, policy and managed catalogs. Fifth, expand the package ecosystem. Longer-term trust-network and process-engine research should be pulled into the product only when real use creates demand.

The long-term bet is that virtual workers will need a workplace. If AI remains mostly a feature inside existing applications, AssistOS may remain a specialized workspace. If persistent agents become a normal part of computing, the need for a neutral control layer grows. The opportunity is asymmetric enough to justify building: the early product can solve useful coordination problems even before the full category exists.

The strategic proposition returns to the first assignment in this book. A person gives a Robot a useful task, can follow its progress and reviews a concrete result. The project retains the work and the decisions needed for next time. Deployment, support and distribution are valuable when they make that experience dependable across people and organizations.

Test the research directions against real work

The research horizon follows from problems encountered in daily work. Can a task be resumed after a change? Can a generated claim be checked against evidence? Can two organizations cooperate while retaining control of their own projects? The product should remain useful while different answers to these questions are tested.

MRP-VM addresses orchestration: representing an objective as an inspectable process with stages, state, dependencies and acceptance conditions. SOP Lang circuits provide a way to express such structure. The purpose is to coordinate model calls, ordinary tools and review steps at the scale of a workflow. The reusable procedures in Chapter 4 give the user-level motivation without requiring this research engine to be installed.

S*LLM, meaning Small, Symbolic, Specialised Language Model, addresses a different scale: explicit language semantics and reasoning within narrower tasks, using SOP circuits alongside LLMs. It should not be conflated with the workflow orchestration role of MRP-VM. A useful result would demonstrate that a particular constraint, relation or claim can be represented and checked more reliably under a defined evaluation. Broader reasoning capability remains an experimental question.

This research supports the practical architectural thesis in the opening: use learned components where language is ambiguous and explicit computation where conditions can be stated and checked. The immediate aim for S*LLM is to improve selected language and reasoning operations alongside LLMs, potentially reducing the size or frequency of model calls needed for those operations. Evidence must come from task-level comparisons, including failures and runtime cost. It is not a claim that the current research already replaces a general-purpose LLM.

Cross-organization collaboration creates another testable problem. Two laboratories may wish to compare results while retaining their own raw data. A proposed workflow sends a bounded request, lets each laboratory run an approved local procedure and returns only the agreed result with its source and run metadata. Ploinky Wormhole explores communication between independently controlled environments. The transport is only one component: the participants still need identity, authorization, a shared interpretation of the request and rules for accepting the returned result. [P6, Chapter 7]

OpenDSU contributes research on provenance, access rights and identity across organizational boundaries. In the laboratory example, these mechanisms could help link an exchanged artifact to its version and authorized source. A signature can establish origin or integrity under its trust assumptions; it cannot establish that the scientific claim is true. Local computation and aggregate exchange also do not, by themselves, guarantee privacy.

WebMeet itself is part product and part research laboratory. The unresolved question is what collaboration becomes when agents are visible participants that can modify shared state. Traditional meeting tools were designed around human audio and video. Agent platforms were designed around tool calls and tasks. Combining them creates new interface and governance questions: how a Robot expresses presence, how actions are attributed, how control is handed over and how meeting decisions become executable processes.

Table 22. Research directions are subordinate to concrete product questions

Term or aspect	Meaning or practical use
MRP-VM / SOP Lang	Can orchestration preserve task state, dependencies, interpreters and acceptance conditions?
S*LLM and SOP circuits	Can narrow language semantics and explicit constraints improve reliability alongside LLMs?
Ploinky Wormhole	Can AssistOS environments communicate directly with explicit identity and local trust policy?
OpenDSU	Can provenance, identity and rights travel with artifacts across organizational boundaries?
WebMeet research	What interaction model works when software workers are visible participants that can change shared state?

These research directions should be judged against the product problem each is intended to solve. A communication prototype needs evidence about authorization and recovery. A reasoning component needs examples of correct behavior, failures and limits. A new process representation needs to show that it preserves useful state and supports reliable changes.

Other exploratory approaches, including vector-symbolic and geometric representations, may become useful for particular memory or reasoning operations. They are options to test. AssistOS does not depend on any of them replacing LLMs, and a successful demonstration on a narrow task should not be generalized beyond the evidence.

A research component belongs in an ordinary Robot workflow when it improves a measured outcome without making the experience harder to operate. Until then, it can remain a separate experiment. The platform's contribution is to provide real tasks, artifacts and evaluation conditions in which such experiments can be challenged.

Appendix A Glossary

Use these definitions when a technical term interrupts the practical account.

Core AssistOS terms

The glossary supports readers who are unfamiliar with AI infrastructure. AssistOS-specific terms use the practical meanings established in the main chapters. General computing terms also have introductory references in the final source list.

Table 23. AssistOS-specific terms

Term or aspect	Meaning or practical use
AssistOS	The overall platform and ecosystem for creating, supervising and collaborating with persistent virtual workers.
Robot	A persistent, personalized virtual worker with a role, agent configuration, home, task history and private sandboxed execution environment.
Robot home	Durable role-specific state comparable in spirit to a Linux home directory; it may include configuration, caches and a browser profile.
RoboTeam	A configured group of Robots working for an individual or team.
Explorer	The plugin-capable web workspace that exposes projects, files, tools, Robots and collaboration surfaces.
WebChat	The AssistOS Copilot and conversational dispatch surface.
WebMeet	The collaborative meeting environment where people and Robots can share screens, chat and modify shared state.
Ploinky	The deployment/runtime system for packaged capabilities.
Ploinky Agent	A deployable capability package; it may contain an AI Agent, tools, services or a complex containerized application.
Task	A bounded unit of Robot work with state, inputs, outputs, limits and history.
Project	The durable workspace in which artifacts, tasks and collaboration accumulate.

The most important distinction to retain is the one among Robot, AI Agent and Ploinky Agent. A Robot is the persistent worker a user delegates to. An AI Agent is reasoning-and-action software. A Ploinky Agent is a deployable capability package that may contain an AI Agent, tools or both. Many public documents use “agent” more broadly; the narrower vocabulary is a product-design choice intended to make the user model stable.

General AI and software terms

Table 24. General AI and software terms

Term or aspect	Meaning or practical use
AI Agent	Software that can pursue a goal by reasoning, choosing actions and using tools on behalf of a user or another program.
Large Language Model (LLM)	A neural language model trained on large text corpora and used for generation, analysis and other language tasks.

Term or aspect	Meaning or practical use
Container	An isolated user-space runtime that packages software and its dependencies while sharing the host kernel.
Sandbox	A restricted execution environment intended to limit what a program can read, write or affect.
API	A defined software interface through which one program requests data or actions from another.
Model Context Protocol (MCP)	An open standard for connecting AI applications to external data sources, tools and workflows.
OpenAI-compatible API	An API that imitates commonly used OpenAI request/response conventions so existing clients can integrate with another service.
WebRTC	A family of browser and real-time communication technologies for direct audio, video and data exchange.
Local-first	A design preference that keeps useful data or computation under local control when practical while still allowing remote services.
Model neutrality	The ability to replace or route among AI models without rebuilding the user's whole work environment.
Provenance	Information about where an artifact came from and which inputs, processes or actors produced it.
Least privilege	The security principle of granting only the permissions necessary for the intended function.

These terms are not unique to AssistOS. The recommended links in the bibliography and glossary point to introductory references rather than assuming that a reader already knows systems engineering vocabulary. A student should be able to read the main chapters first and use the appendix only when a term becomes a barrier.

Appendix B Architecture at a glance

The architecture supports one continuing relationship among people, Robots and projects, from an individual installation to a managed organization.

The smallest useful architecture

A minimal AssistOS installation can be described without a large distributed-systems diagram. The user opens Explorer. WebChat acts as Copilot and dispatch surface. One or more Robots exist as persistent worker identities with homes and task history. Each Robot can use one or more Ploinky Agents. Ploinky starts those capabilities in controlled runtimes. Projects provide the files and artifacts on which work occurs.

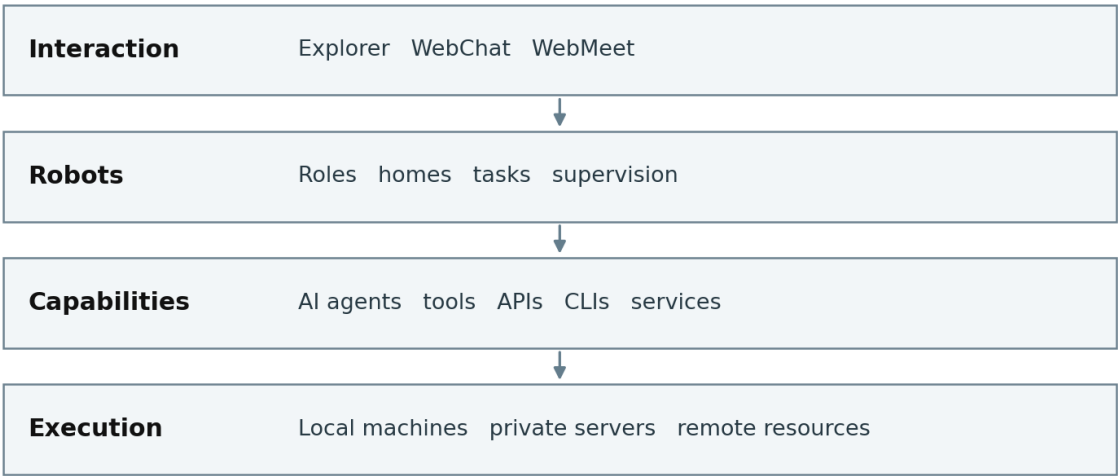


Figure 6. AssistOS architecture at a glance. The product can scale from one machine to managed infrastructure without changing the basic user model.

Figure 6 compresses the architecture into four visible levels. The top is interaction: Explorer, WebChat and WebMeet. The next is the Robot layer, where roles, homes and tasks live. The next is the Ploinky capability layer, where packaged agents and tools run. The bottom is execution infrastructure: the host operating system, containers or lighter sandboxes, local services and optional remote resources. The architecture can grow, but the user story does not need to change.

The main design rule is continuity across replaceable parts. A Robot can survive a restarted container. A project can survive a changed model. A plugin can be replaced without destroying project files. A Ploinky Agent can be upgraded while the task history records which version produced earlier work. This separation is what allows AssistOS to evolve without requiring users to rebuild their working identity every few months.

A company deployment

A company deployment adds policy around the same core. An organizational catalog determines which Ploinky Agents can be enabled. Identity connects humans and Robots to projects. Secret management supplies scoped credentials. Network policy constrains external access. Administrators can inspect task histories and revoke capabilities. Shared servers or workstations can provide execution capacity while Explorer remains the common interface.

Table 25. What an enterprise adds to the same Robot model

Term or aspect	Meaning or practical use
Organizational identity	Connects human users and Robot identities to approved projects and roles.
Capability catalog	Defines which Ploinky Agents and versions can be installed in managed environments.
Secrets service	Provides scoped credentials without storing every secret as a normal file in Robot homes.
Policy layer	Controls project access, network destinations, approval thresholds, model providers and budgets.
Execution pools	Provide managed workstations or servers while preserving the same Robot and task model.
Audit and telemetry	Record significant actions, package versions, costs, failures and administrative interventions.

The table shows that enterprise scale is primarily an addition of governance, not a different product concept. This matters commercially because the individual and enterprise experiences can share the same mental model. The individual user learns to create a Robot, give it a task and inspect results. The company adds rules about which Robots, tasks and capabilities are allowed.

Appendix C Questions for adoption

These questions help turn the ideas in the handbook into an assessable first workflow and deployment.

Questions for a first workflow

A prospective user should not begin by asking which model is strongest. They should begin with the workflow. What work is being delegated? What files or systems are involved? Which actions are reversible? Where does human judgment enter? What constitutes successful output? If these questions cannot be answered at all, the task may be too vague for reliable automation.

Table 26. Questions for a first AssistOS workflow

Term or aspect	Meaning or practical use
Workflow	What recurring digital work do we want the Robot to perform?
Inputs	Which files, websites, applications or data sources are actually required?
Reversibility	Which mistakes can be undone easily, and which actions are consequential?
Credentials	Can the Robot use a scoped identity rather than a human user’s unrestricted credentials?
Human checkpoint	Where should the Robot deliberately pause for judgment or authorization?
Output	What artifact or evidence demonstrates that the task was completed?
Success metric	How will we measure cycle time, human effort, errors and intervention?
Cost boundary	What model or compute budget is acceptable per useful task?
Exit plan	Can the project, artifacts and configuration be exported if the platform or model changes?

The questions in the table can be used in a pilot workshop. They are deliberately phrased in ordinary language. Technical details such as container runtime, MCP transports and model routing can be decided afterward. This ordering keeps the platform focused on the human problem.

Questions for an organization or investor

A company or investor should additionally ask whether the product can be deployed and governed repeatedly. Can a capability be packaged once and installed consistently? Can the organization restrict which projects a Robot can access? Can a Robot be stopped? Can its browser identity be revoked? Can the same project survive a model change? Can the team measure task success and intervention? Can third-party capabilities be evaluated before use?

Early prototypes can demonstrate interaction and architecture. Production claims require reliability measurements, security testing and real deployment evidence. A buyer should establish which features are available in the proposed installation and which remain development or research directions.

Sources and further reading

Project references explain the AssistOS vision and Ploinky runtime. The external sources provide background on interfaces, isolation, collaboration and governance. Illustrative workflows and proposed extensions remain subject to implementation and evaluation.

AssistOS and Ploinky sources

- [P1] [AssistOS — project home](https://www.assistos.org/). <https://www.assistos.org/>
- [P2] [AssistOS Vision](https://www.assistos.org/vision.html). <https://www.assistos.org/vision.html>
- [P3] [AssistOS Whitepaper](https://www.assistos.org/whitepaper/index.html). <https://www.assistos.org/whitepaper/index.html>
- [P4] [AssistOS Developer Guide](https://www.assistos.org/developers.html). <https://www.assistos.org/developers.html>
- [P5] [AssistOS Research Foundation](https://www.assistos.org/research.html). <https://www.assistos.org/research.html>
- [P6] AssistOS The Operating Layer for Sovereign Intelligence. Companion source for this second edition, especially Chapters 4, 5, 7, 8, 9 and 10 and the Strategic Appendix.
<https://www.axiologic.net/content/EN/AssistOS.pdf>
- [P7] [Ploinky — project home](https://www.ploinky.com/). <https://www.ploinky.com/>
- [P8] [Ploinky Agent Specification](https://www.ploinky.com/spec-agent.html). <https://www.ploinky.com/spec-agent.html>
- [P9] [Ploinky WebChat](https://www.ploinky.com/webchat.html). <https://www.ploinky.com/webchat.html>
- [P10] [Ploinky Interfaces](https://www.ploinky.com/interfaces.html). <https://www.ploinky.com/interfaces.html>
- [P11] [Ploinky Architecture](https://www.ploinky.com/architecture.html). <https://www.ploinky.com/architecture.html>

External technical references

- [E1] [Model Context Protocol — official introduction](https://modelcontextprotocol.io/introduction). <https://modelcontextprotocol.io/introduction>
- [E2] [OpenAI API Reference](https://developers.openai.com/api/reference/). <https://developers.openai.com/api/reference/>
- [E3] [Docker Docs — What is a container?](https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/). <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [E4] [Podman Documentation](https://docs.podman.io/en/latest/). <https://docs.podman.io/en/latest/>
- [E5] [MDN — WebRTC API](https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API). https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
- [E6] [LiveKit Agents Documentation](https://docs.livekit.io/agents/). <https://docs.livekit.io/agents/>
- [E7] [OpenAI Codex](https://openai.com/codex/). <https://openai.com/codex/>
- [E8] [OpenAI — Running Codex safely](https://openai.com/index/running-codex-safely/). <https://openai.com/index/running-codex-safely/>
- [E9] [Anthropic — Claude Code CLI reference](https://docs.anthropic.com/en/docs/claude-code/cli-usage). <https://docs.anthropic.com/en/docs/claude-code/cli-usage>
- [E10] [NIST AI Risk Management Framework](https://www.nist.gov/itl/ai-risk-management-framework). <https://www.nist.gov/itl/ai-risk-management-framework>
- [E11] [OWASP Top 10 for Agentic Applications 2026](https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/). <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- [E12] [OWASP State of Agentic AI Security and Governance 2.01](https://genai.owasp.org/resource/state-of-agentic-ai-security-and-governance/). <https://genai.owasp.org/resource/state-of-agentic-ai-security-and-governance/>
- [E13] Apple. Meet the Foundation Models framework. WWDC25, 2025. <https://developer.apple.com/videos/play/wwdc2025/286/>
- [E14] Microsoft. One year of Phi Small language models making big leaps in AI. 2025. <https://azure.microsoft.com/en-us/blog/one-year-of-phi-small-language-models-making-big-leaps-in-ai/>
- [E15] Google DeepMind. AlphaGeometry An Olympiad level AI system for geometry. 2024. <https://deepmind.google/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/>

Introductory glossary references

- [W1] [Software agent](https://en.wikipedia.org/wiki/Software_agent). https://en.wikipedia.org/wiki/Software_agent
- [W2] [Large language model](https://en.wikipedia.org/wiki/Large_language_model). https://en.wikipedia.org/wiki/Large_language_model
- [W3] [Sandbox \(computer security\)](https://en.wikipedia.org/wiki/Sandbox_(computer_security)). [https://en.wikipedia.org/wiki/Sandbox_\(computer_security\)](https://en.wikipedia.org/wiki/Sandbox_(computer_security))
- [W4] [OS-level virtualization](https://en.wikipedia.org/wiki/OS-level_virtualization). https://en.wikipedia.org/wiki/OS-level_virtualization
- [W5] [Application programming interface](https://en.wikipedia.org/wiki/API). <https://en.wikipedia.org/wiki/API>
- [W6] [Model Context Protocol](https://en.wikipedia.org/wiki/Model_Context_Protocol). https://en.wikipedia.org/wiki/Model_Context_Protocol
- [W7] [WebRTC](https://en.wikipedia.org/wiki/WebRTC). <https://en.wikipedia.org/wiki/WebRTC>
- [W8] [Provenance](https://en.wikipedia.org/wiki/Provenance). <https://en.wikipedia.org/wiki/Provenance>
- [W9] [Principle of least privilege](https://en.wikipedia.org/wiki/Principle_of_least_privilege). https://en.wikipedia.org/wiki/Principle_of_least_privilege